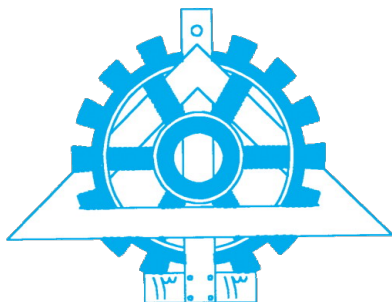




دانشگاه تهران
پردیس دانشکده‌های فنی
دانشکده مهندسی برق و کامپیوتر
گروه مهندسی کامپیوتر



کاهش مدل معنایی ربکا با استفاده از رابطه شبیه‌سازی دوسویه ضعیف زمان‌دار

پروژه کارشناسی مهندسی کامپیوتر گرایش نرم‌افزار

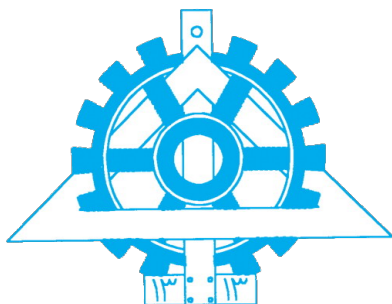
امیرپارسا موبد

استاد راهنما

دکتر فاطمه قاسمی اصفهانی

شهریور ۱۴۰۵





دانشگاه تهران
پردیس دانشکده‌های فنی
دانشکده مهندسی برق و کامپیوتر
گروه مهندسی کامپیوتر



کاهش مدل معنایی ربکا با استفاده از رابطه شبیه‌سازی دوسویه ضعیف زمان‌دار

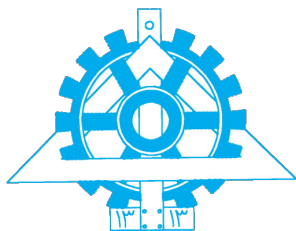
پروژه کارشناسی مهندسی کامپیوتر گرایش نرم‌افزار

امیرپارسا موبد

استاد راهنما

دکتر فاطمه قاسمی اصفهانی

شهریور ۱۴۰۵



دانشگاه تهران
پردیس دانشکده‌های فنی
دانشکده مهندسی برق و کامپیوتر



گواهی دفاع از پروژه کارشناسی

هیأت داوران پروژه کارشناسی آقای / خانم امیرپارسا موبد به شماره دانشجویی ۸۱۰۱۰۰۲۷۱ در رشته مهندسی کامپیوتر - گرایش نرم افزار را در تاریخ با عنوان «کاهش مدل معنایی ربکا با استفاده از رابطه شبیه سازی دوسویه ضعیف زمان دار»

به عدد	به حروف

با نمره نهایی

و درجه

ارزیابی کرد.

ردیف	مشخصات هیأت داوران	نام و نام خانوادگی	مرتبه دانشگاهی	دانشگاه یا مؤسسه	امضا
۱	استاد راهنما	دکتر فاطمه قاسمی اصفهانی	استادیار	دانشگاه تهران	
۲	استاد داور داخلی	دکتر رامتین خسروی		دانشگاه تهران	

نام و نام خانوادگی معاون آموزشی و تحصیلات

تکمیلی پردیس دانشکده‌های فنی:

تاریخ و امضا:

نام و نام خانوادگی معاون تحصیلات تکمیلی و

پژوهشی دانشکده / گروه:

تاریخ و امضا:

تعهدنامه اصالت اثر

باسمه تعالی

اینجانب امیرپارسا موبد تأیید می‌کنم که مطالب مندرج در این پروژه حاصل کار پژوهشی اینجانب است و به دستاوردهای پژوهشی دیگران که در این نوشته از آن‌ها استفاده شده است مطابق مقررات ارجاع گردیده است. این پروژه قبلاً برای احراز هیچ مدرک هم‌سطح یا بالاتری ارائه نشده است.

نام و نام خانوادگی دانشجو: امیرپارسا موبد

تاریخ و امضای دانشجو:

کلیه حقوق مادی و معنوی این اثر
متعلق به دانشگاه تهران است.

چکیده

وارسی مدل با پیمایش فضای حالت یک سامانه، ویژگی‌های مورد انتظار را به صورت خودکار بررسی می‌کند. در مدل‌های هم‌روند و زمان‌دار، ترکیب ترتیب اجرای مؤلفه‌ها، صف پیام‌ها و محدودیت‌های زمانی می‌تواند به رشد سریع فضای حالت و افزایش هزینه تحلیل بینجامد. کاهش مدل، در صورتی که رفتار مورد بررسی را حفظ کند، راهی برای کنترل این مسئله است.

در این پروژه، کاهش مدل معنایی زمان‌دار ربکا بر اساس شبیه‌سازی دوسویه ضعیف زمان‌دار بررسی و پیاده‌سازی شده است. در این روش، تعامل‌های نامرتب با سطح مشاهده انتخاب شده پنهان می‌شوند و حالت‌هایی که از نظر رفتار قابل مشاهده و گذر زمان تمایزپذیر نیستند در یک رده قرار می‌گیرند. سپس از این رده‌ها مدلی کوچک‌تر ساخته می‌شود. ارزیابی روی نمونه‌های موجود نشان داد که این روش می‌تواند ضمن حفظ معیار رفتاری انتخاب شده، اندازه مدل را کاهش دهد. نتیجه پروژه مبنایی برای افزودن این نوع کاهش به فرایند تحلیل مدل‌های ربکا در افرا فراهم می‌کند.

واژگان کلیدی ربکا، افرا، شبیه‌سازی دوسویه ضعیف زمان‌دار، روش‌های صوری، واریسی مدل

فهرست مطالب

ث فهرست تصاویر

ج فهرست جداول

چ فهرست الگوریتم‌ها

۱ فصل ۱: مقدمه

۱ ۱.۱ نرم‌افزار در تعامل با جهان

۲ ۲.۱ هزینه یک فرض زمانی نادرست

۳ ۳.۱ از آزمون تا درستی‌یابی صوری

۴ ۴.۱ مسئله انفجار فضای حالت

۴ ۵.۱ بستر بکا و افرا

۵ ۶.۱ هدف‌ها و پرسش‌های پروژه

۶ ۷.۱ دامنه و دستاوردهای پروژه

۶ ۸.۱ روش کلی ارزیابی

۷ ۹.۱ ساختار پایان‌نامه

۹ فصل ۲: مبانی نظری و تعاریف

۱۰ ۱.۲ از آزمون نرم‌افزار تا درستی‌یابی صوری

۱۱ ۲.۲ واریسی مدل

۱۲ ۳.۲ سیستم‌های واکنشی و بلادرنگ

۴.۲	مدل کنشگر، ربکا و افرا	۱۲
۵.۲	از اتوماتا تا اتوماتای زمان‌دار	۱۴
۶.۲	سامانه‌های انتقال برچسب‌دار و رفتار اجرایی	۱۶
۷.۲	هم‌ارزی‌های رفتاری	۱۷
۸.۲	کنش داخلی و شبیه‌سازی دوسویه ضعیف زمان‌دار	۱۸
۹.۲	بازنمایی متناهی زمان	۲۰
۱۰.۲	هم‌ارزی، افراز و کاهش مدل	۲۰
۲۳	فصل ۳: روش طراحی و پیاده‌سازی	
۱.۳	مقدمه	۲۳
۲.۳	صورت مسئله، ورودی و خروجی	۲۴
۳.۳	جریان پردازش	۲۴
۴.۳	خواندن خروجی افرا	۲۵
۱.۴.۳	قرارداد فایل TTS	۲۵
۲.۴.۳	مرز منبع داده	۲۶
۵.۳	انتخاب مشاهده‌ها و پیش‌پردازش	۲۶
۶.۳	رفع ابهام معنای تأخیر	۲۷
۷.۳	پالایش تأخیر به گام‌های واحد	۲۸
۸.۳	ساخت رابطه‌گذار ضعیف	۲۹
۹.۳	پالایش افراز	۲۹
۱۰.۳	ساخت مدل خارج‌قسمتی و فشرده‌سازی زمان	۳۴
۱۱.۳	وارسی مستقل خروجی	۳۴
۱۲.۳	معماری نرم‌افزار و رابط اجرا	۳۵
۱۳.۳	مصنوعات خروجی	۳۶
۱۴.۳	طرح اعتبارسنجی و تکرار اجرا	۳۶
۱۵.۳	جمع‌بندی	۳۷

فصل ۴:	نتایج و اعتبارسنجی	۳۹
۱.۴	مقدمه	۳۹
۲.۴	ورودی های ارزیابی	۳۹
۳.۴	شرایط اجرا و سنجها	۴۰
۴.۴	نتایج کاهش	۴۱
۵.۴	مطالعه موردی خانه هوشمند	۴۲
۶.۴	حساسیت به معنای زمان و مجموعه مشاهده ها	۴۴
۷.۴	نتایج واری واری رابطه خروجی	۴۴
۸.۴	پوشش آزمون ها	۴۵
۹.۴	زمان اجرا و حافظه	۴۶
۱۰.۴	جمع بندی	۴۶
فصل ۵:	بحث و نتیجه گیری	۴۹
۱.۵	مرور مسیر پروژه	۴۹
۲.۵	پاسخ به پرسش های پروژه	۵۰
۱.۲.۵	خواندن و مستقل کردن خروجی افرا	۵۰
۲.۲.۵	کاهش همراه با واری مستقل	۵۰
۳.۲.۵	اثر معنای زمان و اندازه کاهش	۵۱
۳.۵	تفسیر تصمیم های طراحی	۵۱
۴.۵	جایگاه دستاورد	۵۲
۵.۵	محدودیت ها	۵۲
۶.۵	کارهای آینده	۵۳
۷.۵	جمع بندی نهایی	۵۴
۵۵	کتاب نامه	

واژه‌نامه انگلیسی به فارسی

پنجم

فهرست تصاویر

- ۱.۲ شش نمونه از اتوماتای زمان‌دار؛ برگرفته از شکل ۱ مقاله لیب و همکاران [۱۰]. ۱۵
- ۲.۲ بخش‌هایی از سامانه‌های انتقال برچسب‌دار زمان‌دار متناظر با A_1 ، A_3 و A_4 ؛ برگرفته از شکل ۲ مقاله لیب و همکاران [۱۰]. ۱۷
- ۱.۳ جریان پردازش در ابزار کاهش فضای حالت زمان‌دار. ۲۵
- ۱.۴ فضای حالت خانه هوشمند پیش و پس از کاهش ضعیف زمان‌دار. ۴۳

فهرست جداول

۱.۳	بخش مورد استفاده از قرارداد خروجی فضای حالت افرا	۲۵
۲.۳	مصنوعات تولیدشده برای هر اجرای کاهش	۳۶
۱.۴	ورودی‌ها و هدف هر مورد ارزیابی	۴۰
۲.۴	شمار حالت‌ها، گذارها و نتیجه کاهش	۴۱
۳.۴	نسبت کاهش حالت‌ها و گذارها	۴۲
۴.۴	حساسیت نتیجه به تفسیر تأخیر	۴۴
۵.۴	خلاصه واریسی مستقل مدل‌های کاهش یافته	۴۵
۶.۴	تعداد آزمون‌ها بر حسب هدف	۴۵
۷.۴	زمان و بیشینه حافظه در اجرای ثبت شده	۴۶

فهرست الگوریتم‌ها

۳۰		Tau Closure	۱.۳
۳۱		Weak Successors for One Label	۲.۳
۳۲		Weak Transition Relation	۳.۳
۳۳		Stable Partition Refinement	۴.۳

فصل ۱

مقدمه

۱.۱ نرم افزار در تعامل با جهان

بخش مهمی از نرم افزارهای امروز فقط ورودی ثابتی را دریافت نمی کنند تا پس از محاسبه خاتمه یابند. یک کنترل کننده آسانسور باید در تمام مدت کار ساختمان به درخواست طبقه، بازشدن در، مانع میان در و وضعیت موتور واکنش نشان دهد. یک خدمت دهنده شبکه باید هم زمان پیام چند کاربر، پایان مهلت ها و قطع ارتباط را مدیریت کند. چراغ راهنمایی، کنترل کننده صنعتی و سامانه پایش بیمار نیز پیوسته داده ای از محیط می گیرند و رفتار بعدی خود را بر اساس حالت جاری انتخاب می کنند. این دسته از نرم افزارها نمونه هایی از سیستم های واکنشی^۱ هستند؛ یعنی سامانه هایی که معنای رفتارشان در یک تعامل ادامه دار با محیط شکل می گیرد [۲۴، ۲۵].

در یک برنامه ترتیبی کوچک، یافتن خروجی نادرست برای یک ورودی معمولاً مسیر نسبتاً روشنی برای رفع خطا فراهم می کند. در سامانه واکنشی، نتیجه به تاریخچه رویدادها نیز وابسته است. ممکن است دو پیام جداگانه درست پردازش شوند، اما رسیدن آن ها با ترتیبی خاص به بن بست بینجامد؛ یا خطا تنها پس از تکرار طولانی یک چرخه آشکار شود. اگر چند مؤلفه مستقل و ناهمگام باشند، تعداد ترتیب های ممکن اجرای همان چند رویداد نیز به سرعت افزایش می یابد. بنابراین مسئله فقط درست بودن یک تابع نیست، بلکه باید رفتار همه اجزای مرتبط بررسی شود.

زمان در برخی از این سامانه ها جزئی از تعریف درستی است. در پخش صوت، دیررسیدن یک بسته شاید

¹Reactive System

تنها به افت کیفیت منجر شود؛ اما در ترمز خودرو، تشخیص برخورد یا رهگیری یک هدف، پاسخ درست پس از پایان مهلت دیگر پاسخ قابل قبول نیست. چنین سامانه‌ای سیستم بلادرنگ^۲ نامیده می‌شود. صفت «بلادرنگ» الزاماً به معنای سریع بودن مطلق نیست؛ معنای دقیق‌تر آن این است که رویداد یا پاسخ باید در بازه زمانی تعیین شده رخ دهد. برای نمونه، کنترل دمای یک اتاق می‌تواند مهلتی چندثانیه‌ای داشته باشد، درحالی‌که کنترل یک فرایند فیزیکی سریع به مهلتی بسیار کوتاه‌تر نیاز دارد. آنچه اهمیت دارد پیوند رفتار منطقی و محدودیت زمانی است [۱، ۲].

۲.۱ هزینه یک فرض زمانی نادرست

رخداد سامانه دفاع موشکی پاتریوت در طهران نمونه‌ای روشن از اهمیت همین پیوند است. در ۲۵ فوریه ۱۹۹۱، سامانه نتوانست یک موشک اسکاد ورودی را رهگیری کند و برخورد موشک با محل استقرار نیروها به کشته شدن ۲۸ نفر انجامید. بررسی دفتر حسابرسی عمومی ایالات متحده نشان داد محاسبه زمان در رایانه کنترل تسلیحات بر اثر تبدیل عددی دقت از دست می‌داد. خطای کوچک هر محاسبه با طول مدت کار سامانه انباشته می‌شد؛ سامانه در زمان حادثه بیش از ۱۰۰ ساعت پیوسته کار کرده بود و در نتیجه، محدوده جست‌وجوی هدف را در محل نادرستی محاسبه می‌کرد [۲۷].

این رخداد تنها یک «خطای گردکردن» جدا از طراحی سامانه نبود. پاتریوت در ابتدا برای استقرارهای کوتاه طراحی شده بود و استفاده پیوسته طولانی در فرض‌های اولیه آن قرار نداشت. رفتار نرم‌افزار برای چند ساعت می‌توانست پذیرفتنی باشد، اما همان محاسبه در افق زمانی طولانی ویژگی لازم برای رهگیری را نقض می‌کرد. یک مشخصه صوری که کران خطای زمان یا قراردادن هدف در پنجره رهگیری را برای کل مدت مجاز کار بیان کند، این فرض پنهان را به یک تعهد قابل بررسی تبدیل می‌کند. این گزاره به معنای آن نیست که هر استفاده از روش صوری حتماً حادثه را پیش‌بینی می‌کرد؛ اگر محدوده کار یا ویژگی مورد انتظار اشتباه مشخص شود، اثبات نیز همان فرض اشتباه را معتبر می‌گیرد. ارزش روش صوری در واداشتن طراح به بیان دقیق چنین فرض‌هایی و بررسی همه حالت‌های مدل‌شده است.

شکست نخستین پرواز آریان ۵ در سال ۱۹۹۶ جنبه دیگری از مسئله را نشان می‌دهد. موشک حدود ۴۰ ثانیه پس از آغاز پرواز از مسیر خارج شد و از بین رفت. گزارش رسمی آژانس فضایی اروپا علت را خطاهای

^۲Real-Time System

مشخصه و طراحی در نرم‌افزار سامانه مرجع اینرسی دانست؛ دو واحد افزونه نیز به دلیل اجرای نرم‌افزار یکسان تقریباً هم‌زمان از کار افتادند. آزمون‌های انجام‌شده مسیر واقعی آریان ۵ را به اندازه کافی بازنمایی نکرده بودند و در نتیجه پیش شرط نامعتبر تبدیل عددی کشف نشد [۷]. این مثال یادآوری می‌کند که افزونگی سخت‌افزاری در برابر خطای مشترک طراحی محافظت کافی ایجاد نمی‌کند و پوشش آزمون نیز به سناریوهای انتخاب‌شده محدود است.

۳.۱ از آزمون تا درستی‌یابی صوری

آزمون نرم‌افزار همچنان بخش ضروری فرایند مهندسی است: اجرای واقعی می‌تواند خطای پیاده‌سازی، ناسازگاری محیط و مسئله‌ای را آشکار کند که در مدل انتزاعی حضور ندارد. محدودیت بنیادی آزمون این است که هر مورد آزمون تنها یک یا چند اجرای مشخص را مشاهده می‌کند. در سامانه‌های هم‌رند و زمان‌دار، فضای ترتیب پیام‌ها، انتخاب‌ها و مقدارهای زمانی ممکن است چنان بزرگ باشد که پیمایش دستی یا اجرای همه آزمون‌ها عملی نباشد.

در درستی‌یابی صوری^۳، مدل سامانه و ویژگی مورد انتظار با زبان دقیق ریاضی بیان می‌شوند. سپس یک روش استدلالی یا الگوریتمی بررسی می‌کند آیا ویژگی برای رفتارهای مدل برقرار است یا خیر. واریسی مدل^۴ یکی از این روش‌هاست که فضای حالت مدل را به طور خودکار جست‌وجو می‌کند و در صورت نقض ویژگی معمولاً یک مثال نقض ارائه می‌دهد [۶، ۳]. تضمین حاصل نه درباره هر جزئیات جهان واقعی، بلکه درباره مدل، مشخصه و فرض‌های تعریف‌شده است؛ از این رو، آزمون و درستی‌یابی صوری مکمل یکدیگرند.

کاربرد روش‌های صوری به پژوهش دانشگاهی محدود نمانده است. مهندسان Amazon Web Services از سال ۲۰۱۱ مشخصه‌نویسی صوری و واریسی مدل با TLA+ را برای مسائل دشوار طراحی سامانه‌های توزیع‌شده بحرانی به کار گرفته‌اند [۱۳]. در Meta، تحلیل‌گر ایستای Infer با استدلال نمادین و منطق ریاضی، مسیرهای متعدد برنامه را بدون اجرای آن بررسی می‌کند و در زمان گزارش رسمی، هر ماه صدها خطای احتمالی پیش از انتشار رفع می‌شدند؛ البته این ابزار تضمین کامل درستی تابعی ارائه نمی‌کند [۱۱]. پروژه Everest در مایکروسافت نیز مؤلفه‌های رمزنگاری، تجزیه‌گرها و پروتکل‌های ارتباط امن را همراه با اثبات‌های ماشین‌بررسی‌شده برای

^۳Formal Verification

^۴Model Checking

استفاده عملی توسعه داده است [۲۶]. این نمونه‌ها طیفی از روش‌های صوری را نشان می‌دهند: از تحلیل خودکار برای یافتن رده‌ای از خطاها تا اثبات ویژگی‌های مشخص یک مؤلفه.

۴.۱ مسئله انفجار فضای حالت

پوشش همه رفتارهای یک مدل هزینه دارد. اگر هر مؤلفه چند حالت داشته باشد، ترکیب هم‌زمان چند مؤلفه می‌تواند تعداد بسیار بیشتری حالت سراسری بسازد. صف پیام، مقدار متغیرها، ترتیب اجرای کنشگرها و وضعیت ساعت‌ها نیز این فضا را گسترش می‌دهند. این پدیده انفجار فضای حالت^۵ نام دارد و یکی از موانع اصلی کاربرد واریسی مدل است [۳، ۶].

با این حال، همه جزئیات مدل برای هر پرسش تحلیلی اهمیت یکسان ندارند. فرض کنید هدف، بررسی توالی فرمان‌های قابل مشاهده یک کنترل‌کننده باشد. گام‌های داخلی محاسبه یا پیام‌هایی که در سطح انتزاع مورد نظر دیده نمی‌شوند، ممکن است مسیر اجرا را طولانی کنند بی‌آنکه مشاهده بیرونی تازه‌ای بسازند. اگر بتوان حالت‌هایی را که از دید رفتار مورد توجه تمایزپذیر نیستند شناسایی و ادغام کرد، تحلیل روی مدلی کوچک‌تر انجام می‌شود. شرط اساسی آن است که کاهش، همان جنبه رفتاری را که قرار است بررسی شود حفظ کند. در سامانه زمان‌دار، صرفاً حذف کنش داخلی کافی نیست. دو اجرا ممکن است از نظر توالی پیام‌های قابل مشاهده یکسان باشند، اما مدت متفاوتی داشته باشند. شبیه‌سازی دوسویه ضعیف زمان‌دار اجازه می‌دهد کنش‌های داخلی نادیده گرفته شوند، درحالی‌که رفتار قابل مشاهده و مقدار زمان سپری شده حفظ می‌شود [۲۲، ۱۰]. رده‌های هم‌ارزی حاصل می‌توانند مبنای ساخت یک مدل خارج‌قسمتی کوچک‌تر باشند.

۵.۱ بستر ربکا و افرا

ربکا^۶ زبانی کنشگرمحور برای مدل‌سازی سامانه‌های هم‌روند و توزیع‌شده است. هر شیء واکنشی حالت و صف پیام خود را دارد و از طریق ارسال ناهمگام پیام با دیگر اشیا ارتباط برقرار می‌کند. ربکای زمان‌دار^۷

^۵State-Space Explosion

^۶Rebeca

^۷Timed Rebeca

سازوکارهایی برای تأخیر، مهلت و زمان محاسبه به این مدل می افزاید [۲۴، ۲۵، ۲۳]. این ساختار برای بیان شهودی یک سامانه واکنشی مناسب است، اما ترکیب صف ها، کنشگرها و زمان همان مسئله رشد فضای حالت را ایجاد می کند.

افرا^۸ محیط مدل سازی و تحلیل خانواده ربکا است و با گردآوردن کامپایلر، واریسی گر مدل و ابزارهای نمایش، فضای حالت حاصل را برای تحلیل در اختیار کاربر می گذارد [۱۴، ۱۵]. ورودی پروژه حاضر کد منبع ربکا نیست، بلکه سامانه انتقال زمان داری است که این زنجیره با قالب `statespace` تولید می کند. بنابراین مسئله در مرز میان تولید مدل معنایی و تحلیل آن قرار دارد: چگونه می توان با انتخاب تعامل های قابل مشاهده، مدل را بر اساس شبیه سازی دوسویه ضعیف زمان دار کاهش داد و هم زمان شاهدهی قابل واریسی برای حفظ رفتار ارائه کرد؟

۶.۱ هدف ها و پرسش های پروژه

هدف اصلی این پروژه تبدیل تعریف و شبه کد کاهش ضعیف زمان دار به ابزاری اجرایی برای فضای حالت افراس است. این پایان نامه الگوریتم هم ارزی تازه ای پیشنهاد نمی کند؛ کار آن تطبیق تعریف موجود با قرارداد واقعی خروجی افرا، پیاده سازی فرایند کاهش و فراهم کردن شواهد قابل تکرار برای درستی خروجی است. پروژه سه پرسش را دنبال می کند:

۱. آیا می توان خروجی زمان دار افرا را بدون ویرایش دستی خواند و به یک نمایش مستقل از رابط کاربری افرا تبدیل کرد؟

۲. آیا می توان پس از پنهان سازی تعامل های انتخاب نشده، یک مدل خارج قسمتی کوچک تر ساخت و رابطه آن با ورودی را به طور مستقل واریسی کرد؟

۳. تصمیم معنایی درباره جمع شدن تأخیرهای جدا شده با کنش داخلی چه اثری بر هم ارزی نمونه ها دارد و روش روی ورودی های موجود چه مقدار کاهش ایجاد می کند؟

برای پاسخ به این پرسش ها، هدف های اجرایی زیر تعیین شده اند:

- استخراج و مستند سازی بخش مورد نیاز از قرارداد فایل TTS افرا؛

^۸Afra

- پیاده‌سازی پنهان‌سازی، پالایش تأخیر، ساخت گذار ضعیف و پالایش افراز^۹؛
- ساخت و فشرده‌سازی سامانه انتقال خارج‌قسمتی^{۱۰} همراه با نگاشت حالت‌های ورودی؛
- توسعه واریسی‌گری مستقل برای آزمودن رابطه مدل تحویلی و ورودی؛
- ثبت مصنوعات، سنجه‌ها و آزمون‌های لازم برای تکرار ارزیابی.

۷.۱ دامنه و دستاوردهای پروژه

دامنه پیاده‌سازی به سامانه‌های انتقال متناهی با زمان گسسته و تأخیر صحیح نامنفی محدود است. ابزار خروجی TTS افرا را می‌خواند و از FTTS، زمان چگال، کامپایل مستقیم مدل ربکا و ادغام به‌صورت افزونه رابط کاربری افرا پشتیبانی نمی‌کند. معنای پیش‌فرض، تأخیر چندواحدی را به گام‌های واحد می‌شکند تا تأخیرهای جداشده با کنش داخلی جمع‌پذیر باشند؛ خوانش سخت‌گیرانه شبه‌کد نیز برای مقایسه حفظ شده است. فصل ۳ این تصمیم را دقیق‌تر توضیح می‌دهد.

دست‌آورد پروژه یک ابزار مستقل جاوا با رابط خط فرمان، خواننده مقاوم در برابر ویژگی خاص فایل واقعی افرا، خط لوله کاهش قابل ردیابی و واریسی‌گر جداگانه است. نتیجه اصلی به‌صورت JSON بدون اتلاف، نگاشت افراز، نمایش سازگار با افرا، نمودار و سنجه‌های اجرا ذخیره می‌شود [۱۲]. ارزش این کار در پیوند زدن تعریف نظری با قرارداد داده و نیازهای مهندسی زنجیره ربکا و افراست؛ ادعای مینیمال یا کانونی بودن مدل خروجی و نیز ادعای مقیاس‌پذیری صنعتی مطرح نمی‌شود.

۸.۱ روش کلی ارزیابی

پیاده‌سازی در سه سطح بررسی شده است. آزمون‌های واحد، اجزای خواندن فایل، معنای گذار ضعیف، پالایش افراز، ساخت خارج‌قسمت و واریسی رابطه را می‌سنجند. آزمون‌های انتها به انتها برنامه بسته‌بندی شده را از

⁹Partition Refinement

¹⁰Quotient Transition System

بیرون اجرا و مصنوعات، کدهای خروج و قطعی بودن نتیجه را بررسی می کنند. در نهایت، چهار ورودی شامل یک نمونه کوچک و سه بیان از مطالعه موردی خانه هوشمند ارزیابی شده اند. دو نمونه خانه هوشمند، تأخیر ده واحدی را به ترتیب به $3 + \tau + 7$ و $7 + \tau + 3$ می شکنند تا حساسیت روش به معنای زمان آشکار شود. سنجه های فصل ۴ مستقیماً از خروجی ثبت شده همین اجراها گزارش می شوند.

۹.۱ ساختار پایان نامه

فصل ۲ پیش نیازهای درستی یابی صوری، واری مدل، سامانه های واکنشی و زمان دار، ربکا و افرا، سامانه های انتقال و تعریف شبیه سازی دوسویه ضعیف زمان دار را ارائه می کند. فصل ۳ قرارداد ورودی، تصمیم های معنایی، مراحل الگوریتم، معماری پیاده سازی و روش اعتبارسنجی را شرح می دهد. فصل ۴ نتیجه کاهش، حساسیت معنایی، پوشش آزمون ها، زمان اجرا و تحلیل نتایج را گزارش می کند. در پایان، فصل ۵ به پرسش های پروژه پاسخ می دهد، دستاوردها و محدودیت ها را جمع بندی می کند و مسیرهای ادامه کار را پیشنهاد می دهد.

فصل ۲

مبانی نظری و تعاریف

این فصل مفاهیمی را معرفی می‌کند که برای دنبال کردن مسئله و روش این پروژه لازم‌اند. متن با فرض آشنایی خواننده با مباحث عمومی دوره کارشناسی مهندسی کامپیوتر نوشته شده است، نه با فرض آشنایی او با روش‌های صوری یا سیستم‌های زمان‌دار. از این رو، هر موضوع ابتدا به صورت شهودی و سپس، هر جا در ادامه به دقت ریاضی نیاز باشد، به صورت صوری بیان می‌شود. هدف ارائه یک مرجع جامع برای هریک از این حوزه‌ها نیست؛ خواننده‌ای که با یک موضوع آشناست می‌تواند از بخش مربوط به آن عبور کند.

برخی اصطلاحات این حوزه در منابع مختلف به تعریف‌های یکسانی اشاره نمی‌کنند. برای مثال، دامنه زمان ممکن است گسسته یا چگال باشد، اتوماتای زمان‌دار ممکن است با شرط پذیرش یا در قالب اتوماتای ایمنی تعریف شود، و شبیه‌سازی دوسویه زمان‌دار می‌تواند قوی یا ضعیف باشد [۱، ۲، ۱۰، ۲۲]. این تفاوت‌ها صرفاً واژگانی نیستند و می‌توانند رفتار مدل و نتیجه تحلیل را تغییر دهند. بنابراین، تعریف‌های صوری این فصل قرارداد دقیق پایان‌نامه را تعیین می‌کنند؛ به ویژه، تعریف قوی از لیب و همکاران و تعریف ضعیف از رینیر و سانیه گرفته شده است و نباید این دو به جای یکدیگر به کار روند.

مسیر فصل از چرایی درستی یابی نرم‌افزار آغاز می‌شود و پس از معرفی واریسی مدل و سیستم‌های بلادرنگ، مدل کنشگر، ربکا و افرا را معرفی می‌کند. سپس اتوماتای زمان‌دار، معنای عملیاتی آن، هم‌ارزی‌های رفتاری و تعریف شبیه‌سازی دوسویه ضعیف زمان‌دار ارائه می‌شوند. در پایان نیز ارتباط این تعریف با افراز فضای حالت و کاهش مدل توضیح داده می‌شود.

۱.۲ از آزمون نرم‌افزار تا درستی‌یابی صوری

نرم‌افزار معمولاً با مجموعه‌ای از آزمون‌های واحد، یکپارچه و سیستمی ارزیابی می‌شود. هر آزمون، اجرای سامانه را برای یک ورودی و شرایط مشخص مشاهده می‌کند و می‌تواند وجود یک خطا را آشکار سازد. با این حال، موفقیت مجموعه‌ای متناهی از آزمون‌ها به‌تنهایی نشان نمی‌دهد که سامانه در همه ورودی‌ها و همه ترتیب‌های اجرای ممکن درست رفتار می‌کند. این محدودیت به‌ویژه در سامانه‌های هم‌روند و واکنشی مهم است؛ زیرا زمان‌بندی متفاوت رویدادها می‌تواند تعداد بسیار زیادی رفتار ممکن پدید آورد [۳].

روش‌های صوری^۱ مجموعه‌ای از زبان‌ها، مدل‌ها و فنون مبتنی بر ریاضیات‌اند که برای توصیف دقیق، تحلیل و ساخت سامانه‌های نرم‌افزاری و سخت‌افزاری به کار می‌روند. استفاده از این روش‌ها ابهام زبان طبیعی را کاهش می‌دهد و امکان استدلال نظام‌مند درباره رفتار سامانه را فراهم می‌کند. در این پایان‌نامه، جریان به‌کارگیری روش‌های صوری در سه بخش مکمل در نظر گرفته می‌شود [۶، ۳، ۲۴]:

۱. مشخصه‌نویسی صوری^۲: نیازمندی‌ها، ساختار یا رفتار مورد انتظار سامانه با یک زبان دقیق ریاضی بیان می‌شود؛

۲. درستی‌یابی صوری: بررسی می‌شود که مدل یا پیاده‌سازی، مشخصه صوری تعیین‌شده را برآورده می‌کند یا نه؛

۳. پالایش صوری^۳ یا توسعه صوری^۴: مشخصه انتزاعی طی گام‌های کنترل‌شده به طراحی یا پیاده‌سازی دقیق‌تر تبدیل می‌شود، به‌گونه‌ای که ویژگی‌های لازم در هر گام حفظ شوند.

این سه بخش الزاماً خطی و مستقل از یکدیگر نیستند؛ نتیجه درستی‌یابی ممکن است به اصلاح مشخصه بینجامد و هر گام پالایش نیز تعهدهای درستی‌یابی تازه‌ای ایجاد کند.

درستی‌یابی نرم‌افزار^۵ مفهومی گسترده‌تر از اجرای آزمون‌هاست و بررسی می‌کند که یک مصنوع نرم‌افزاری نیازمندی‌های تعیین‌شده را برآورده می‌کند یا نه. در درستی‌یابی صوری، خود سامانه و ویژگی مورد انتظار با زبان ریاضی توصیف می‌شوند و درستی رابطه میان آن‌ها با استدلال ریاضی یا یک روش الگوریتمی بررسی می‌شود.

¹Formal Method

²Formal Specification

³Formal Refinement

⁴Formal Development

⁵Software Verification

بنابراین، یک مسئله درستی‌یابی صوری معمولاً سه جزء دارد: مدلی از سامانه، مشخصه‌ای صوری از رفتار مطلوب یا نامطلوب، و روشی برای اثبات یا رد برقراری آن مشخصه [۶، ۲۵].

نتیجه درستی‌یابی همواره نسبت به مدل و مشخصه‌ای است که تحلیل شده‌اند. اثبات درست‌بودن یک مدل، خودبه‌خود خطای پیاده‌سازی، فرض نادرست محیط، یا نیازمندی‌ای را که در مشخصه نیامده است حذف نمی‌کند. به همین دلیل، آزمون و درستی‌یابی صوری جانشین کامل یکدیگر نیستند: آزمون رفتار پیاده‌سازی واقعی را در اجراهای انتخاب‌شده می‌سنجد، درحالی‌که درستی‌یابی صوری درباره همه رفتارهای مدل، در محدوده فرض‌های صریح آن، استدلال می‌کند [۳].

دو دسته رایج از ویژگی‌ها عبارت‌اند از ویژگی‌های ایمنی و زنده‌بودن. ویژگی ایمنی بیان می‌کند که «رخداد بدی اتفاق نمی‌افتد»؛ برای نمونه، دو فرایند هم‌زمان وارد یک بخش بحرانی نمی‌شوند. ویژگی زنده‌بودن بیان می‌کند که «رخداد مطلوبی سرانجام اتفاق می‌افتد»؛ برای نمونه، هر درخواست پذیرفته‌شده سرانجام پاسخی دریافت می‌کند. بیان دقیق این ویژگی‌ها ممکن است با منطق‌های زمانی انجام شود، اما جزئیات آن‌ها برای تعریف‌های این فصل لازم نیست [۳].

۲.۲ واریسی مدل

واریسی مدل یکی از روش‌های خودکار درستی‌یابی صوری برای سامانه‌هایی است که رفتارشان به صورت حالت و گذار مدل شده است. اگر M مدل سامانه و φ ویژگی مورد نظر باشد، پرسش اصلی به شکل

$$M \models \varphi$$

نوشته می‌شود: آیا همه رفتارهای مدل M مشخصه φ را ارضا می‌کنند؟ الگوریتم واریسی مدل فضای حالت قابل‌دسترسی را به طور نظام‌مند بررسی می‌کند. اگر ویژگی برقرار نباشد، ابزار معمولاً یک مثال نقض ارائه می‌کند؛ یعنی اجرایی از مدل که چگونگی نقض ویژگی را نشان می‌دهد [۶، ۳].

مزیت واریسی مدل، خودکاربودن و پوشش همه رفتارهای مدلی است که در محدوده تحلیل قرار دارد. مانع اصلی آن انفجار فضای حالت است: تعداد حالت‌های مدل ممکن است با افزایش مؤلفه‌ها، متغیرها یا ترتیب‌های ممکن اجرا به سرعت رشد کند. در چنین شرایطی، حتی اگر هر گام بررسی ساده باشد، نگهداری و پیمایش همه

حالت‌ها پرهزینه یا ناممکن می‌شود. کاهش مدلی که رفتارهای مرتبط با ویژگی مورد بررسی را حفظ کند، یکی از راه‌های مقابله با این مسئله است [۶، ۲۴].

۳.۲ سیستم‌های واکنشی و بلادرنگ

یک سیستم واکنشی در تعامل پیوسته با محیط قرار دارد: ورودی دریافت می‌کند، حالت داخلی خود را تغییر می‌دهد و در پاسخ، خروجی یا رویدادی تولید می‌کند. پروتکل‌های ارتباطی، کنترل‌کننده‌ها و سامانه‌های توزیع‌شده نمونه‌هایی از این الگو هستند. در یک سامانه هم‌روند، چند مؤلفه می‌توانند به صورت مستقل پیشرفت کنند و ترتیب نسبی کنش‌های آن‌ها بخشی از رفتار سامانه است [۲۴، ۲۵].

در سیستم بلادرنگ، درست بودن پاسخ فقط به مقدار منطقی آن وابسته نیست و زمان تولید پاسخ نیز اهمیت دارد. برای نمونه، فرمان درست یک کنترل‌کننده اگر پس از مهلت لازم صادر شود می‌تواند بی‌فایده باشد. آلور و دیل برای مدل کردن چنین محدودیت‌هایی، رویدادهای یک رد را با زمان حقیقی وقوع آن‌ها همراه کردند و صورت اولیه اتوماتای زمان‌دار را ارائه دادند [۱].

برای تحلیل چنین سامانه‌ای باید گذر زمان نیز در مدل قابل مشاهده باشد. یک مدل صرفاً ترتیبی ممکن است نشان دهد که کنش a پیش از b رخ داده است، اما بیان نمی‌کند فاصله زمانی میان آن دو چقدر بوده یا آیا یک مهلت رعایت شده است. اتوماتای زمان‌دار این کمبود را با افزودن ساعت و قیده‌های زمانی به اتوماتا جبران می‌کند [۱، ۲، ۱۰].

۴.۲ مدل کنشگر، ربکا و افرا

مدل کنشگر^۶ سامانه هم‌روند را به مجموعه‌ای از موجودیت‌های مستقل تقسیم می‌کند. هر کنشگر حالت داخلی خود را در اختیار دارد و با دریافت و ارسال ناهمگام پیام با کنشگرهای دیگر تعامل می‌کند. این جداسازی، واحد هم‌روندی را خود کنشگر قرار می‌دهد و برای مدل کردن سامانه‌های توزیع‌شده و واکنشی مناسب است [۲۴، ۲۵].

^۶ Actor Model

نام ربکا کوتاه‌شده Reactive Objects Language است. ربکا زبانی کنشگرمحور برای مدل‌سازی و درستی‌یابی سامانه‌های هم‌روند و توزیع‌شده است. این زبان با هدف نزدیک کردن مدل‌سازی صوری به ساختارهای آشناتر مهندسی نرم‌افزار طراحی شده و نحو آن به جاوا شباهت دارد. در ربکا، یک reactive class الگوی ساخت اشیای واکنشی است و هر نمونه آن یک rebec نامیده می‌شود. هر ربکا حالت محصور، رشته اجرای مستقل و صف پیام خود را دارد. دریافت یک پیام اجرای message server متناظر را فعال می‌کند و ارتباط میان ربکاها از طریق ارسال ناهمگام و نامسدودکننده پیام انجام می‌شود [۲۴، ۲۵].

تعریف صوری نحو و معنای ربکا و روش‌های درستی‌یابی ترکیبی آن در رساله دکتری مرجان سیرجانی ارائه شده است. در آن رساله، ساختار محصور و ارتباط ناهمگام ربکاها مبنایی برای تجزیه مدل، تجرید و کاهش فضای حالت قرار می‌گیرد. مقاله سیرجانی و همکاران نیز این چارچوب را با سه جزء مدل رفتاری، زبان مشخصه و روش تحلیل توضیح می‌دهد و درستی تجرید پیشنهادی خود را با یک رابطه شبیه‌سازی ضعیف استدلال می‌کند [۲۴، ۲۵].

ربکای زمان‌دار ربکا را با سازوکارهای بیان تأخیرهای محاسباتی و شبکه‌ای و مهلت زمانی گسترش می‌دهد تا رفتار سامانه‌های توزیع‌شده ناهمگام زمان‌دار مدل شود. معنای این زبان به صورت صوری تعریف شده است و مدل حاصل می‌تواند برای تحلیل رفتار زمانی به کار رود [۲۳].

افرا محیط یکپارچه مدل‌سازی و درستی‌یابی خانواده ربکا است. افرا امکانات ویرایش مدل و ویژگی، اجرای واریسی مدل، نمایش نتیجه و مشاهده مثال نقض را یکجا فراهم می‌کند و اجزایی مانند کامپایلر ربکا و Rebeca Model Checker را در یک جریان کاری گرد هم می‌آورد [۱۴]. کد منبع اجزای اصلی این زنجیره، از جمله افرا، کامپایلر و واریسی‌گر مدل، در مخازن رسمی گروه پژوهشی ربکا منتشر شده است [۱۹، ۱۵، ۱۶، ۱۷].

در این پروژه، اهمیت افرا فقط به رابط کاربری آن محدود نیست: مدل معنایی تولیدشده برای یک مدل ربکای زمان‌دار، فضای حالتی را فراهم می‌کند که کاهش رفتاری روی آن تعریف می‌شود. بنابراین، ربکا زبان مدل‌سازی، افرا و واریسی‌گر مدل ابزار ساخت و تحلیل مدل معنایی، و شبیه‌سازی دوسویه ضعیف زمان‌دار معیار تشخیص حالت‌های قابل ادغام‌اند [۱۴، ۲۲].

۵.۲ از اتوماتا تا اتوماتای زمان‌دار

خواننده در درس زبان‌های صوری و نظریه اتوماتا در دوره کارشناسی با ایده اصلی اتوماتا آشنا شده است: مجموعه‌ای از حالت‌ها و گذارهای برچسب‌دار که نحوه تغییر حالت را مشخص می‌کنند. در این فصل تنها همان بخش از این مفهوم بازخوانی می‌شود که برای مدل‌های زمان‌دار لازم است. در ادبیات اتوماتای زمان‌دار معمولاً به جای «حالت کنترلی» از واژه مکان و به جای یال از واژه سویچ استفاده می‌شود [۹، ۱، ۱۰].

اتوماتای زمان‌دار، گراف متناهی اتوماتای کلاسیک را با مجموعه‌ای متناهی از ساعت‌های نامنفی گسترش می‌دهد. همه ساعت‌ها با آهنگی یکسان پیش می‌روند، اما هر گذار می‌تواند تعدادی از آن‌ها را مستقل از بقیه به صفر بازنشانی کند. قیده‌های روی ساعت‌ها تعیین می‌کنند یک گذار چه زمانی مجاز است و ماندن در یک مکان تا چه زمانی امکان دارد [۱، ۲].

تعریف ۱.۵.۲ (اتوماتای زمان‌دار). یک اتوماتای زمان‌دار چندتایی

$$A = (L, l, \Sigma, C, I, E)$$

است که در آن L مجموعه‌ای متناهی از مکان‌ها، $l \in L$ مکان آغازین، Σ الفبای متناهی کنش‌ها و C مجموعه‌ای متناهی از ساعت‌هاست. تابع $I : L \rightarrow \mathcal{B}(C)$ به هر مکان یک ناورد نسبت می‌دهد. همچنین

$$E \subseteq L \times \mathcal{B}(C) \times \Sigma \times 2^C \times L$$

مجموعه سویچ‌هاست و $\mathcal{B}(C)$ مجموعه قیده‌های ساعتی روی C را نشان می‌دهد. هر عضو E یک سویچ به شکل زیر است:

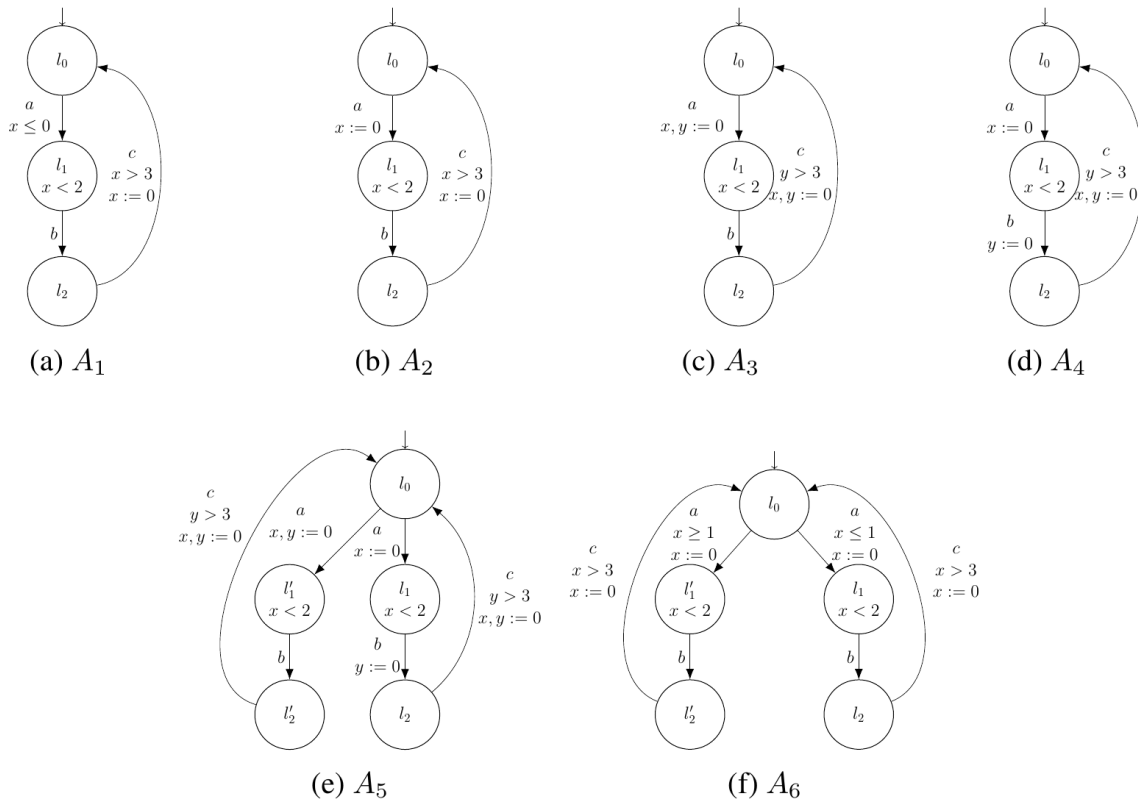
$$(l, g, a, R, l').$$

این سویچ تنها هنگامی قابل اجراست که شرط نگهبان g برقرار باشد. اجرای آن کنش a را انجام می‌دهد، ساعت‌های مجموعه R را صفر می‌کند و مکان فعال را به l' تغییر می‌دهد. ناوردای $I(l)$ نیز قیدی است که تا هنگام حضور در مکان l باید برقرار بماند [۲، ۱۰].

دامنه زمان می‌تواند گسسته، مانند $\mathbb{N}_{\geq 0}$ ، یا چگال، مانند $\mathbb{R}_{\geq 0}$ ، باشد. انتخاب دامنه بخشی از مدل‌سازی

است و باید با سامانه و روش تحلیل سازگار باشد. در مدل چگال، میان هر دو لحظه متمایز لحظه‌های دیگری وجود دارند؛ در مدل گسسته، زمان با گام‌های شمارا پیش می‌رود [۱۰، ۱].

شکل ۱.۲ شش نمونه از اتوماتای زمان‌دار را نشان می‌دهد. دایره‌ها مکان‌ها هستند. روی یک سویچ ممکن است کنش، شرط نگهدار و بازنشانی ساعت نوشته شود؛ برای مثال، $x := 0$ ساعت x را هنگام عبور صفر می‌کند. قید نوشته‌شده داخل یک مکان، ناوردای آن مکان است. این نمونه‌ها همچنین نشان می‌دهند که دو اتوماتا با ساختار ظاهری مشابه می‌توانند به دلیل تفاوت در نگهدار یا بازنشانی ساعت، رفتار زمانی متفاوتی داشته باشند [۱۰].



شکل ۱.۲: شش نمونه از اتوماتای زمان‌دار؛ برگرفته از شکل ۱ مقاله لیب و همکاران [۱۰].

تعریف ۲.۵.۲ (مقداردهی ساعت). برای مجموعه ساعت‌های C و دامنه زمان $\mathbb{T}$ ، یک مقداردهی ساعت تابع $u : C \rightarrow \mathbb{T}$ است. مقدار $u(c)$ زمان سپری‌شده از آخرین بازنشانی ساعت c را نشان می‌دهد. نماد $u + d$ مقداردهی‌ای است که برای هر $c \in C$ مقدار $u(c) + d$ را برمی‌گرداند. اگر $R \subseteq C$ باشد، $[R \mapsto 0]u$ یعنی مقداردهی ساعت‌های عضو R را صفر و مقدار بقیه ساعت‌ها را بدون تغییر نگه می‌دارد. همچنین $u \models \phi$

u قید ساعتی ϕ را ارضا می‌کند [۱۰].

۶.۲ سامانه‌های انتقال برچسب‌دار و رفتار اجرایی

برای تحلیل یک مدل باید میان نمایش نحوی آن و رفتار حاصل از اجرای آن تمایز گذاشت. اتوماتای زمان‌دار یک نمایش متناهی و فشرده است، اما معنای اجرایی آن مشخص می‌کند از هر وضعیت دقیقاً چه کنش یا چه مقدار تأخیر ممکن است. این معنا با سامانه انتقال برچسب‌دار^۷ بیان می‌شود [۳، ۱۰].

تعریف ۱.۶.۲ (سامانه انتقال برچسب‌دار). یک سامانه انتقال برچسب‌دار چندتایی $S = (Q, q_0, \Lambda, \rightarrow)$ است که Q مجموعه حالت‌ها، $q_0 \in Q$ حالت آغازین، Λ مجموعه برچسب‌ها و $Q \times \Lambda \times Q \rightarrow \subseteq$ رابطه گذار است. نماد $q \xrightarrow{\mu} q'$ بیان می‌کند که سامانه می‌تواند با گذار دارای برچسب μ از q به q' برود [۳].

یک اجرا دنباله‌ای متناوب از حالت‌ها و گذارهاست که از حالت آغازین یا یک حالت مشخص شروع می‌شود. با کنارگذاشتن خود حالت‌ها، دنباله برچسب‌های این اجرا به دست می‌آید. اگر تنها کنش‌های قابل مشاهده نگه داشته شوند، این دنباله رد^۸ نامیده می‌شود. تفاوت میان اجرا و رد مهم است: چند اجرای متفاوت ممکن است رد یکسانی تولید کنند، درحالی‌که انتخاب‌ها و انشعاب‌های داخلی آن‌ها یکسان نباشد [۳، ۱].

تعریف ۲.۶.۲ (سامانه انتقال برچسب‌دار زمان‌دار). فرض کنید $A = (L, l_0, \Sigma, C, I, E)$ یک اتوماتای زمان‌دار روی دامنه زمان $\mathbb{T}$ باشد. سامانه انتقال برچسب‌دار زمان‌دار^۹ متناظر با آن چندتایی

$$S_A = (Q, q_0, \Sigma \cup \mathbb{T}, \rightarrow)$$

است که $Q = L \times (C \rightarrow \mathbb{T})$ و $q_0 = (l_0, [C \mapsto \circ])$ است. دو نوع گذار دارد:

- گذار تأخیر $(l, u) \xrightarrow{d} (l, u + d)$ برای $d \in \mathbb{T}$ ، مشروط به اینکه ناوردای مکان در طول این تأخیر نقض

نشود؛

⁷Labeled Transition System

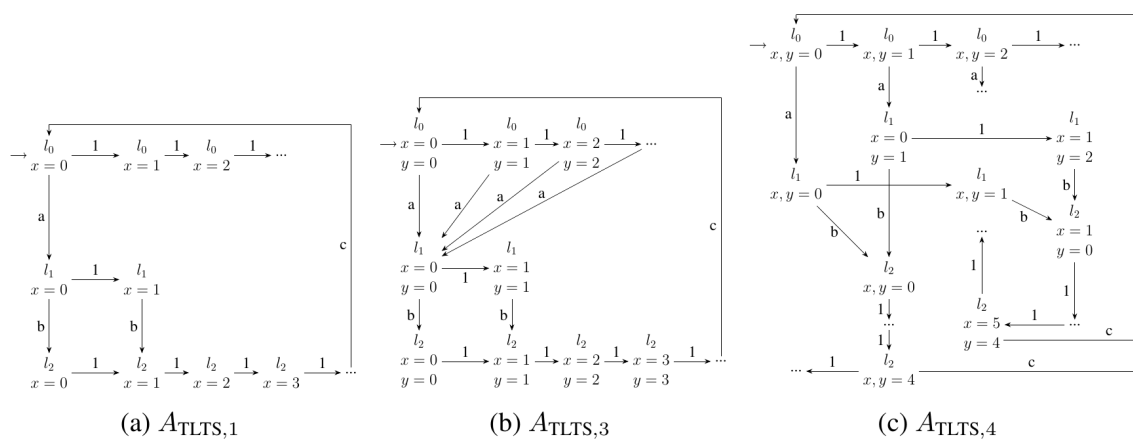
⁸Trace

⁹Timed Labeled Transition System

- گذار کنش $(l, u) \xrightarrow{a} (l', [R \mapsto \circ]u)$ ، هرگاه سوییچ (l, g, a, R, l') وجود داشته باشد، $u \models g$ و مقداردهی مقصد ناوردای l' را ارضا کند.

[۱۰]

در نتیجه، یک حالت معنایی فقط نام مکان نیست و مقدار همه ساعت ها را نیز در بر می گیرد. حتی اتوماتایی با تعداد متناهی مکان می تواند سامانه انتقال زمان داری با بی نهایت حالت داشته باشد. شکل ۲.۲ بخشی از سامانه های انتقال متناظر با سه اتوماتای شکل ۱.۲ را در زمان گسسته نشان می دهد. یال های با برچسب ۱ گذر یک واحد زمان و یال های با برچسب های a ، b و c اجرای کنش را نشان می دهند [۱۰].



شکل ۲.۲: بخشی از سامانه های انتقال برچسب دار زمان دار متناظر با A_1 ، A_3 و A_4 ؛ برگرفته از شکل ۲ مقاله لیب و همکاران [۱۰].

۷.۲ هم ارزی های رفتاری

برای مقایسه یک مشخصه و پیاده سازی، یا برای تشخیص حالت های قابل ادغام در یک مدل، باید روشن باشد «رفتار یکسان» دقیقاً به چه معناست. برابری ردها تنها مجموعه دنباله های قابل مشاهده را مقایسه می کند و ممکن است محل انتخاب میان رفتارهای آینده را نادیده بگیرد. شبیه سازی رابطه ای جهت دار است: هر گام یک سامانه باید در سامانه دیگر قابل تقلید باشد. شبیه سازی دوسویه^{۱۰} این شرط را در هر دو جهت برقرار می کند و در نتیجه ساختار انشعاب رفتار را نیز حفظ می کند [۳، ۱۰].

¹⁰Bisimulation

تعریف ۱.۷.۲ (شبیه‌سازی دوسویه زمان‌دار قوی). فرض کنید $(Q_1, q_1, \Sigma \cup \mathbb{T}, \rightarrow_1)$ و $S_2 = S_1 = (Q_2, q_2, \Sigma \cup \mathbb{T}, \rightarrow_2)$ دو سامانه انتقال زمان‌دار باشند. رابطه $R \subseteq Q_1 \times Q_2$ یک شبیه‌سازی دوسویه زمان‌دار قوی است، هرگاه برای هر $(p, q) \in R$ و هر $\mu \in \Sigma \cup \mathbb{T}$ شروط زیر برقرار باشند:

۱. اگر $p \xrightarrow{\mu}_1 p'$ ، حالتی مانند q' وجود داشته باشد که $q \xrightarrow{\mu}_2 q'$ و $(p', q') \in R$ ؛

۲. اگر $q \xrightarrow{\mu}_2 q'$ ، حالتی مانند p' وجود داشته باشد که $p \xrightarrow{\mu}_1 p'$ و $(p', q') \in R$.

اگر افزون بر این، $(q_1, q_2) \in R$ باشد، دو سامانه شبیه‌سازی دوسویه زمان‌دار قوی اند [۵، ۱۰].

واژه «قوی» در این تعریف یعنی هر گذار باید جداگانه و با همان برچسب پاسخ داده شود. اگر مدل گذار داخلی داشته باشد، آن گذار نیز از این قاعده مستثنا نیست. این حساسیت برای مقایسه دقیق دو مدل مفید است، اما هنگام کاهش مدل می‌تواند جزئیات داخلی‌ای را حفظ کند که از دید ناظر اهمیتی ندارند [۱۰، ۲۲].

۸.۲ کنش داخلی و شبیه‌سازی دوسویه ضعیف زمان‌دار

همه گذارهای یک مدل الزاماً برای محیط قابل مشاهده نیستند. در این پایان‌نامه، τ برچسب یک کنش داخلی^{۱۱} و Σ مجموعه کنش‌های قابل مشاهده است. نماد ε نیز منحصرراً برای کلمه تهی به کار می‌رود. رویکرد ضعیف اجازه می‌دهد تعدادی گذار داخلی پیرامون یک کنش قابل مشاهده یا در خلال سپری شدن زمان نادیده گرفته شوند. با این حال، مقدار کل زمان سپری شده باید حفظ شود [۲۲].

برای بیان تعریف، یک اجرای متناهی را به صورت

$$q_0 \xrightarrow{\mu_1} q_1 \xrightarrow{\mu_2} \dots \xrightarrow{\mu_n} q_n$$

در نظر بگیرید که هر μ_i یا یک کنش از $\Sigma_\tau = \Sigma \cup \{\tau\}$ یا مدتی از $\mathbb{R}_{\geq 0}$ است. $\text{Untimed}(w)$ دنباله‌ای است که با حذف تأخیرها و کنش‌های τ از برچسب‌های اجرای w به دست می‌آید؛ $\text{Duration}(w)$ نیز مجموع برچسب‌های زمانی آن اجراست [۲۲].

¹¹Internal Action

تعریف ۱.۸.۲ (رابطه گذار ضعیف). فرض کنید $S = (Q, q_0, \Sigma_\tau, \rightarrow)$ یک سامانه انتقال زمان‌دار باشد که در آن $Q \times (\Sigma_\tau \cup \mathbb{R}_{\geq 0}) \times Q \rightarrow$ رابطه گذار ضعیف را با $\xrightarrow{d}_W$ نشان می‌دهیم:

- برای $d \in \mathbb{R}_{\geq 0}$ ، رابطه $q \xrightarrow{d}_W q'$ برقرار است، اگر اجرایی w از q به q' وجود داشته باشد و

$$\text{Untimed}(w) = \varepsilon, \quad \text{Duration}(w) = d;$$

- برای $a \in \Sigma$ ، رابطه $q \xrightarrow{a}_W q'$ برقرار است، اگر اجرایی w از q به q' وجود داشته باشد و

$$\text{Untimed}(w) = a, \quad \text{Duration}(w) = 0.$$

بنابراین، یک گام ضعیف کنش می‌تواند گذارهای داخلی پیش و پس از دقیقاً یک کنش قابل مشاهده را در خود جمع کند، بدون آنکه زمانی بگذرد. یک گام ضعیف زمانی نیز می‌تواند چند گذار زمانی و داخلی را ترکیب کند، به شرط آنکه هیچ کنش قابل مشاهده‌ای رخ ندهد و مجموع تأخیرها دقیقاً d باشد [۲۲].

تعریف ۲.۸.۲ (شبیه‌سازی دوسویه ضعیف زمان‌دار). فرض کنید $S_1 = (Q_1, q_{0,1}, \Sigma_\tau, \rightarrow_1)$ و $S_2 = (Q_2, q_{0,2}, \Sigma_\tau, \rightarrow_2)$ دو سامانه انتقال زمان‌دار باشند. رابطه $R \subseteq Q_1 \times Q_2$ یک شبیه‌سازی دوسویه ضعیف زمان‌دار است، هرگاه $(q_{0,1}, q_{0,2}) \in R$ و برای هر $(p, q) \in R$ و هر $\mu \in \Sigma \cup \mathbb{R}_{\geq 0}$ شروط زیر برقرار باشند:

$$۱. \text{ اگر } p \xrightarrow{\mu}_{W,1} p', \text{ حالتی مانند } q' \text{ وجود داشته باشد که } q \xrightarrow{\mu}_{W,2} q' \text{ و } (p', q') \in R;$$

$$۲. \text{ اگر } q' \xrightarrow{\mu}_{W,2} q, \text{ حالتی مانند } p' \text{ وجود داشته باشد که } p \xrightarrow{\mu}_{W,1} p' \text{ و } (p', q') \in R.$$

وجود چنین رابطه‌ای را با $S_1 \approx_W S_2$ نشان می‌دهیم [۲۲].

تفاوت اصلی تعریف ضعیف با تعریف قوی در این است که سامانه پاسخ‌دهنده مجبور نیست تک‌تک گام‌های داخلی سامانه نخست را بازتولید کند. آنچه باید یکسان بماند، کنش قابل مشاهده، زمان کل سپری شده و امکان ادامه رفتار از حالت‌های مقصد است. این نکته اجازه می‌دهد حالت‌هایی که تنها در جزئیات داخلی تفاوت دارند در یک رده رفتاری قرار گیرند، بی‌آنکه تمایز زمانی قابل مشاهده از میان برود [۲۲].

۹.۲ بازنمایی متناهی زمان

در زمان چگال، حتی یک اتوماتای زمان‌دار کوچک می‌تواند بی‌نهایت مقداردهی ساعت و در نتیجه بی‌نهایت حالت معنایی داشته باشد. برای تحلیل الگوریتمی باید مجموعه‌های نامتناهی از مقداردهی‌ها به شکل متناهی نمایش داده شوند. دو بازنمایی شناخته شده برای این منظور گراف ناحیه و گراف zone هستند [۲، ۸، ۱۰].

در گراف ناحیه، مقداردهی‌هایی که همه قیده‌های مرتبط را یکسان ارزیابی می‌کنند و ترتیب عبور بخش کسری ساعت‌هایشان از مرزهای صحیح مشابه است در یک کلاس هم‌ارزی قرار می‌گیرند. تعداد این کلاس‌ها برای یک اتوماتای معین متناهی است؛ از این رو گراف ناحیه امکان تصمیم‌گیری درباره برخی مسائل را فراهم می‌کند، اما با بزرگ شدن ثابت‌های زمانی یا تعداد ساعت‌ها می‌تواند بسیار بزرگ شود [۲، ۵].

یک zone مجموعه‌ای از مقداردهی‌های ساعت است که با یک هم‌بندی از قیده‌های ساعتی نمایش داده می‌شود. گراف‌های zone معمولاً چند ناحیه را در یک حالت نمادین جمع می‌کنند و در عمل از گراف ناحیه فشرده‌ترند. ماتریس کران تفاضل یا DBM نمایش متداول این قیده‌هاست [۸، ۴]. با این حال، فشرده‌سازی ممکن است اطلاعاتی مانند اثر یک بازنشانی ساعت بر انشعاب‌های آینده را پنهان کند؛ بنابراین مناسب بودن یک بازنمایی نمادین به ویژگی رفتاری مورد بررسی وابسته است. لیب و همکاران برای واریسی شبیه‌سازی دوسویه زمان‌دار، گراف‌های zone را با ساعت‌های مجازی گسترش داده‌اند تا بخشی از این اطلاعات حفظ شود [۱۰].

۱۰.۲ هم‌ارزی، افراز و کاهش مدل

شبیه‌سازی دوسویه یک رابطه هم‌ارزی است؛ یعنی حالت‌ها را می‌توان بر اساس آن به رده‌های جدا از هم تقسیم کرد. چنین تقسیم‌بندی‌ای افراز^{۱۲} نام دارد. در هر بلوک افراز، حالت‌هایی قرار می‌گیرند که مطابق تعریف انتخاب شده از دید رفتاری تمایز پذیر نیستند [۳، ۲۲].

با جایگزین کردن هر بلوک با یک حالت و انتقال گذارها میان بلوک‌ها، مدل خارج‌قسمتی به دست می‌آید. اگر افراز نسبت به شبیه‌سازی دوسویه ضعیف زمان‌دار پایدار باشد، ادغام حالت‌های یک بلوک کنش‌های قابل مشاهده، زمان کل و ساختار انشعاب مورد نظر تعریف را حفظ می‌کند. هدف کاهش مدل، ساختن چنین نمایش کوچک‌تری است؛ نه حذف دلخواه حالت‌ها. الگوریتم‌های پالایش افراز معمولاً از یک افراز اولیه شروع می‌کنند

¹²Partition

و هر بلوکی را که اعضایش نتوانند گام‌های ضعیف یکدیگر را پاسخ دهند می‌شکنند. این فرایند تا رسیدن به افزای پایدار ادامه می‌یابد [۳].

این زنجیره مفهومی جایگاه مسئله را روشن می‌کند: واریسی مدل با رشد فضای حالت محدود می‌شود؛ مدل‌های بلادرنگ علاوه بر کنش، گذر زمان را نیز ثبت می‌کنند؛ و شبیه‌سازی دوسویه ضعیف زمان‌دار معیاری فراهم می‌کند تا جزئیات داخلی پنهان شوند، درحالی‌که رفتار زمانی قابل مشاهده حفظ می‌شود [۳، ۱، ۲۲]. فصل‌های بعد باید مشخص کنند مدل ورودی این پروژه دقیقاً چه ساختاری دارد، تعریف بالا چگونه در الگوریتم به کار رفته است و کاهش حاصل با چه شواهدی ارزیابی می‌شود.

فصل ۳

روش طراحی و پیاده‌سازی

۱.۳ مقدمه

هدف این پروژه ساخت ابزاری است که فضای حالت زمان‌دار صادرشده از افرا را دریافت کند، تعامل‌های خارج از مجموعه مورد توجه کاربر را پنهان سازد، حالت‌ها را بر اساس شبیه‌سازی دوسویه ضعیف زمان‌دار افراز کند و سامانه خارج‌قسمتی حاصل را تحویل دهد. ورودی الگوریتم یک سامانه انتقال متناهی است، نه کد منبع ربکا؛ بنابراین، تولید فضای حالت همچنان بر عهده زنجیره ابزار ربکا و افراس است. پیاده‌سازی حاصل با نام awtr در مخزن پروژه نگهداری می‌شود [۱۲].

روش انجام کار ترکیبی از طراحی الگوریتم، مهندسی نرم‌افزار و ارزیابی تجربی است. ابتدا قرارداد واقعی فایل خروجی افرا از روی تولیدکننده و خواننده‌های رسمی آن استخراج شد. سپس شبکه کد مبنای پروژه به تعریفی اجرایی تبدیل شد و ابهام مربوط به تجزیه تأخیرها با استفاده از نمونه‌های پذیرش آشکار و به صورت یک انتخاب قابل آزمایش پیاده‌سازی شد. پس از ساخت افراز، سامانه انتقال خارج‌قسمتی تولید و با واریسی گری مستقل بررسی شد. در پایان، رفتار ابزار با آزمون‌های واحد، آزمون‌های انتهابه‌انتها^۱ و چهار ورودی ارزیابی سنجیده شد. این فصل روش و تصمیم‌های طراحی را شرح می‌دهد و فصل ۴ نتایج ثبت‌شده را گزارش می‌کند.

¹End-to-End Testing

۲.۳ صورت مسئله، ورودی و خروجی

فرض کنید سامانه انتقال زمان‌دار متناهی پس از خواندن فایل ورودی به صورت

$$T = (S, s_0, Act \cup \{\tau\} \cup \mathbb{N}_{>0}, \rightarrow)$$

نمایش داده شود. S مجموعه حالت‌ها، s_0 حالت آغازین، Act مجموعه تعامل‌های نام‌دار و $d \in \mathbb{N}_{>0}$ برچسب گذر دقیقاً d واحد زمان است. کاربر در هر اجرا زیرمجموعه‌ای مانند $O \subseteq Act$ را به عنوان تعامل‌های قابل مشاهده تعیین می‌کند. همه تعامل‌های دیگر به τ نگاشت می‌شوند و سپس افراز پایدار متناظر با تعریف ۲.۸.۲ محاسبه می‌شود.

خروجی اصلی فقط یک پاسخ بله یا خیر نیست. ابزار باید سه شیء مرتبط را تولید کند: نگاشت هر حالت ورودی به رده هم‌ارزی، سامانه انتقال خارج‌قسمتی، و رابطه‌ای که نشان دهد مدل ورودی و خروجی مطابق تعریف انتخاب‌شده رفتار یکسان دارند. علاوه بر این، برای تکرارپذیری هر اجرا، مجموعه تعامل‌های قابل مشاهده، هش ورودی، نسخه ابزار، شناسه کامیت، زمان اجرا و اندازه‌های پیش و پس از کاهش ثبت می‌شوند [۱۲].

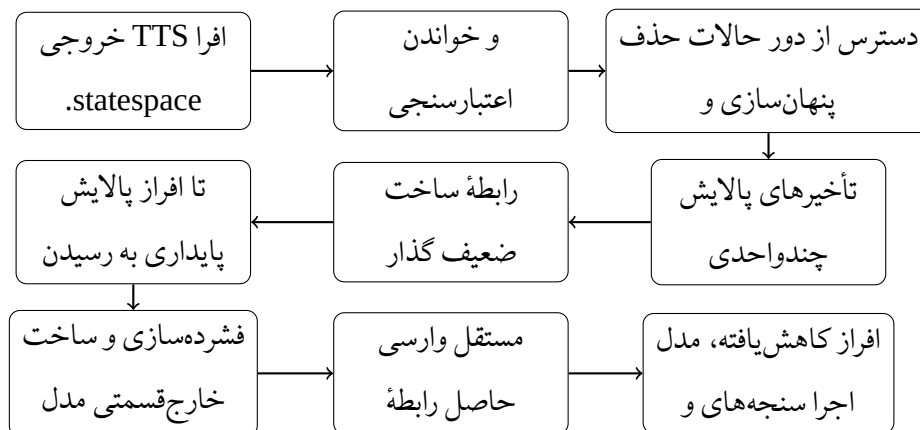
دامنه روش به زمان گسسته و تأخیرهای صحیح نامنفی محدود است. ورودی‌های زمان چگال، معنای FTS، واریسی شبیه‌سازی دوسویه قوی و کامپایل مستقیم کد ربکا در این پروژه پیاده‌سازی نشده‌اند. همچنین، هدف ساخت مدلی کوچک‌تر و واریسی شده است؛ مینیمال یا یکتا بودن مدل خروجی ادعا نمی‌شود.

۳.۳ جریان پردازش

شکل ۱.۳ مسیر داده را از فایل افرا تا مصنوعات خروجی نشان می‌دهد. مرز میان خواندن فایل و هسته الگوریتم صریح نگه داشته شده است؛ از این رو، تغییر منبع داده، منطق پنهان‌سازی^۲، پالایش افراز یا ساخت خارج‌قسمت را تغییر نمی‌دهد.

این تفکیک دو فایده دارد. نخست، هر مرحله قرارداد مشخصی دارد و می‌توان آن را جداگانه آزمود. دوم، واریسی‌گر خروجی از پیاده‌سازی پالایش افراز استفاده مجدد نمی‌کند؛ در نتیجه، خطای مشترک در محاسبه رابطه

²Hiding



شکل ۱.۳: جریان پردازش در ابزار کاهش فضای حالت زمان‌دار.

ضعیف و واریسی آن به‌سادگی پنهان نمی‌شود.

۴.۳ خواندن خروجی افرا

۱.۴.۳ قرارداد فایل TTS

ساختار فایل statespace با بررسی تولیدکننده رسمی RMC، طرح داده افرا و دو خواننده رسمی فضای حالت تعیین شد [۱۵، ۱۷، ۲۱، ۲۰]. قواعدی که مستقیماً بر پیاده‌سازی اثر گذاشتند در جدول ۱.۳ آمده‌اند.

جدول ۱.۳: بخش مورد استفاده از قرارداد خروجی فضای حالت افرا

تفسیر در ابزار	جزء فایل
شناسه حالت به‌صورت رشته‌ای و بدون تفسیر ساختار داخلی آن نگهداری می‌شود.	state/@id
حالت آغازین است؛ گزینه initial-state- امکان جایگزینی صریح آن را فراهم می‌کند.	state نخستین عنصر
ویژگی title هویت پایه تعامل است؛ صورت owner.title نیز برای رفع ابهام قابل انتخاب است.	messageserver
مدت سپری‌شده است و باید یک عدد صحیح نامنفی باشد.	time/@value
برای ردیابی نگه داشته می‌شوند، اما در مدت گذار دخالت داده نمی‌شوند.	shift و executionTime

یک ناسازگاری ظاهری در خروجی واقعی وجود دارد: تولیدکننده RMC عنصر ریشه `transitionsystem` را باز می‌کند، اما برچسب پایانی آن را نمی‌نویسد. بنابراین، فایل واقعی از دید یک تجزیه‌گر سخت‌گیر، XML خوش‌ساخت نیست. خواننده این پروژۀ انتهایی فایل را بررسی می‌کند و فقط در صورت نبودن برچسب پایانی، آن را به جریان ورودی اضافه می‌کند. به این ترتیب فایل بدون ویرایش دستی مصرف می‌شود و فایل‌های دارای برچسب پایانی نیز پذیرفته می‌شوند [۱۷، ۲۰، ۱۲].

تنها خروجی‌های TTS برای کاهش زمانی مناسب‌اند؛ زیرا در این حالت، گذر زمان به‌صورت عناصر مستقل `time` ثبت می‌شود. در FTTS زمان در گذارهای پیام ادغام می‌شود و یال تأخیر مستقلی برای الگوریتم حاضر وجود ندارد. خواننده همچنین شناسۀ تکراری حالت، مقصد یا مبدأ تعریف نشده، زمان منفی یا اعشاری، پیام فاقد ویژگی‌های لازم، و وجود هم‌زمان برچسب پیام و زمان روی یک گذار را رد می‌کند. پردازش DTD و موجودیت خارجی نیز غیرفعال شده است.

۲.۴.۳ مرز منبع داده

خواننده فایل یکی از پیاده‌سازی‌های رابط `TransitionSystemSource` است. این رابط تنها یک عمل برای ساخت نمایش خام سامانه دارد. نمایش خام، نام فرستنده، مالک و عنوان پیام و نیز اطلاعات زمانی جانبی را حفظ می‌کند. سپس مرحله جداگانه‌ای آن را به سامانه انتقال مورد استفاده الگوریتم تبدیل می‌کند. یک منبع درون حافظه‌ای نیز همین رابط را پیاده‌سازی می‌کند و آزمون قرارداد نشان می‌دهد هر دو منبع به مدل دامنه و نتیجه کاهش یکسان می‌رسند. این مرز امکان افزودن یک آداپتور درون‌فرایندی برای افرا را فراهم می‌کند، بی‌آنکه هسته کاهش به کلاس‌های رابط کاربری یا XML وابسته شود [۱۲].

۵.۳ انتخاب مشاهده‌ها و پیش‌پردازش

مجموعه تعامل‌های قابل مشاهده در زمان اجرا و با گزینه `observable`—تعیین می‌شود. یک نام ساده مانند `getSense` با عنوان پیام در هر مالک تطبیق داده می‌شود؛ صورت مقید مانند `controller.getSense` فقط همان مالک را می‌پذیرد. تطبیق به بزرگی و کوچکی حروف حساس است. این تصمیم از ادغام ناخواسته دو شناسه متمایز جلوگیری می‌کند و با هویت ثبت‌شده در خروجی افرا سازگار است [۲۰، ۲۱].

اگر α یک تعامل نام دار و O مجموعه انتخابی کاربر باشد، تابع پنهان سازی به صورت زیر عمل می کند:

$$h_O(\alpha) = \begin{cases} \alpha & \alpha \in O, \\ \tau & \alpha \notin O. \end{cases}$$

گذر بدون برچسب و گذر زمان صفر نیز به τ تبدیل می شوند. عنوان هایی که افرا با پیشوند τ به عنوان ادامه اجرای یک پیام ثبت کرده است، با یک نام ساده قابل مشاهده نمی شوند. پس از پنهان سازی، فقط زیرسامانه قابل دسترس از حالت آغازین نگه داشته می شود. ابزار نام های قابل مشاهده ای را که با هیچ گذار ورودی تطبیق ندارند نیز گزارش می کند؛ زیرا یک خطای تایپی در این گزینه می تواند ناخواسته همه تعامل ها را پنهان کند.

۶.۳ رفع ابهام معنای تأخیر

شبه کد مبنای پروژه برای محاسبه گذار ضعیف با مدت d ، یک یال منفرد با برچسب d را میان دو بستر τ قرار می دهد. در مقابل، توضیح همان تابع و تعریف ۱.۸.۲ وجود اجرایی را ملاک می دانند که محتوای قابل مشاهده ندارد و مجموع مدت آن دقیقاً d است [۲۲]. این دو خوانش در حضور چند یال تأخیر و گام های داخلی یکسان نیستند.

سه نمونه خانه هوشمند این تفاوت را آشکار می کنند. در نمونه پایه، یک دوره با یال منفرد ده واحدی نمایش داده شده است. همان دوره در نمونه دوم به هفت واحد، یک گام داخلی و سه واحد، و در نمونه سوم به سه واحد، یک گام داخلی و هفت واحد شکسته شده است:

$$\begin{aligned} s_0 &\xrightarrow{10} s_1, \\ t_0 &\xrightarrow{7} t_1 \xrightarrow{\tau} t_2 \xrightarrow{3} t_3, \\ u_0 &\xrightarrow{3} u_1 \xrightarrow{\tau} u_2 \xrightarrow{7} u_3. \end{aligned}$$

طبق ملاک پذیرش پروژه، این سه نمایش باید رفتار زمانی قابل مشاهده یکسانی داشته باشند. خوانش یال منفرد نمی تواند گذر هفت واحدی نمونه دوم را با یال ده واحدی نمونه نخست پاسخ دهد. در خوانش مبتنی بر جمع پذیری

زمان^۳، یال ده‌واحدی را می‌توان در لحظه‌های میانی مشاهده کرد و مجموع مدت اجرا معیار تطبیق است؛ بنابراین سه نمایش قابل مقایسه می‌شوند.

پیاده‌سازی خوانش دوم را با گزینه پیش‌فرض unit به کار می‌گیرد و خوانش لفظی شبه‌کد را با گزینه strict حفظ می‌کند. آزمون حساسیت معنایی نشان می‌دهد نمونه‌های خانه هوشمند در حالت نخست هم‌ارزند و در حالت دوم هم‌ارز نیستند. به این ترتیب تصمیم معنایی در کد پنهان نشده و قابل بازیابی باقی مانده است. جمع‌پذیری زمان قرارداد معنایی تأییدشده پروژه و مبنای نتایج اصلی است؛ خوانش strict فقط برای مقایسه و سنجش حساسیت معنایی حفظ می‌شود.

۷.۳ پالایش تأخیر به گام‌های واحد

برای پیاده‌سازی جمع‌پذیری زمان، هر یال $s \xrightarrow{d} t$ با $d > 1$ ، پیش از محاسبه رابطه ضعیف به زنجیره‌ای از d یال یک‌واحدی تبدیل می‌شود:

$$s \xrightarrow{1} v_1 \xrightarrow{1} v_2 \xrightarrow{1} \dots \xrightarrow{1} v_{d-1} \xrightarrow{1} t.$$

حالت‌های v_i مصنوعی‌اند و از حالت‌های اظهارشده در ورودی متمایز نگه داشته می‌شوند. پس از این بازنویسی، رسیدن ضعیف با مدت d برابر با ترکیب d باره رسیدن ضعیف یک‌واحدی است. در نتیجه، مجموعه برچسب‌های مورد نیاز پالایش افراز متناهی می‌ماند و الگوریتم لازم نیست همه مدت‌های قابل دستیابی را جداگانه تولید کند.

هزینه این روش افزودن

$$\sum_{e \in E_{time}} (d_e - 1)$$

حالت میانی است. ابزار برای تعداد حالت‌های افزوده‌شده سقف قابل تنظیم دارد و در صورت عبور از آن، به جای ادامه کنترل‌نشده، اجرا را با پیام خطا متوقف می‌کند. این راه‌حل برای مدل‌های ارزیابی حاضر کافی است، اما در مدل‌هایی با ثابت‌های زمانی بزرگ می‌تواند عامل اصلی مصرف حافظه باشد.

³Time Additivity

۸.۳ ساخت رابطه‌گذار ضعیف

پس از پالایش زمانی، برای هر حالت ابتدا بستار τ با یک پیمایش عمقی محاسبه می‌شود. سپس برای هر برچسب قابل مشاهده و برچسب تأخیر واحد، مجموعه مقصدهای $\tau^* \ell \tau^*$ ساخته می‌شود. بستار τ نیز به‌عنوان حرکت تهی در پالایش شرکت می‌کند؛ این همان حالت مدت صفر در تعریف ۱.۸.۲ است. حذف این حالت می‌تواند شاخه داخلی منتهی به بن‌بست را از دید الگوریتم پنهان کند.

در پیاده‌سازی، حالت‌ها یک‌بار شماره‌گذاری و مجموعه مقصدها با BitSet نگهداری می‌شوند. بنابراین، محاسبه اجتماع مقصدها و مقایسه امضاها از عملیات بیتی استفاده می‌کند. ترتیب حالت‌ها از ورودی و ترتیب برچسب‌ها به‌صورت مرتب‌شده حفظ می‌شود تا اجرای دوباره روی ورودی یکسان مصنوعات متنی یکسانی تولید کند [۱۲].

۹.۳ پالایش افراز

الگوریتم با افراز یک بلوکی $P_* = \{S\}$ آغاز می‌شود. برای افراز جاری P و برچسب ℓ ، مجموعه بلوک‌های قابل دستیابی از s برابر است با

$$R_P(s, \ell) = \{[t]_P \mid s \xrightarrow{\ell}_W t\}.$$

امضای هر حالت از بلوک فعلی آن، مجموعه بلوک‌های قابل دستیابی برای تمام برچسب‌ها، و بلوک‌های موجود در بستار τ ساخته می‌شود:

$$\text{Sig}_P(s) = \left([s]_P, (R_P(s, \ell))_{\ell \in L}, \{[t]_P \mid t \in \tau^*(s)\} \right).$$

حالت‌های یک بلوک که امضای متفاوت دارند از یکدیگر جدا می‌شوند. وجود $[s]_P$ در امضا تضمین می‌کند دوره‌های بعدی فقط بلوک‌های موجود را بشکنند و دو بلوک جدا شده دوباره ادغام نشوند. فرایند زمانی پایان می‌یابد که هیچ امضای تازه‌ای باعث شکستن بلوک نشود. افراز حاصل پایدار است و رده‌های شبیه‌سازی دوسویه ضعیف

زمان‌دار را برای معنای ورودی محاسبه می‌کند.

الگوریتم‌های ۱.۳ تا ۴.۳ شبه‌کد اصلی این فرایند را در چهار مرحله نشان می‌دهند. جداسازی این مراحل روشن می‌کند که رابطه ضعیف چگونه از گذارهای مستقیم ساخته می‌شود و افراز در هر دور دقیقاً بر چه اساسی شکسته می‌شود. در این الگوریتم‌ها، $C_\tau(s)$ بستر کنش داخلی از حالت s ، L مجموعه برچسب‌های غیر از τ و W رابطه گذار ضعیف است. در حالت پیش‌فرض پیاده‌سازی، پالایش واحدی پیش از اجرای این مراحل انجام می‌شود و تنها برچسب زمانی موجود در L برابر یک واحد است؛ در خوانش سخت‌گیرانه، L مدت‌های ثبت شده روی یال‌های ورودی را نیز در بر می‌گیرد.

الگوریتم ۱.۳ بستر داخلی یک حالت را محاسبه می‌کند. این بستر شامل همه حالت‌هایی است که با صفر یا چند گذار داخلی دسترس‌پذیرند. افزودن خود s به مجموعه آغازین، حرکت داخلی تهی را نیز پوشش می‌دهد. مجموعه *visited* مانع بررسی دوباره حالت‌ها می‌شود و پایان پیمایش را در حضور چرخه‌های τ تضمین می‌کند.

الگوریتم ۱.۳ Tau Closure

Require: A transition system $T = (S, L \cup \{\tau\}, \rightarrow)$ and a state $s \in S$

Ensure: All states reachable from s through zero or more τ transitions

```

1: visited  $\leftarrow \{s\}$ 
2: stack  $\leftarrow [s]$ 
3: while stack is not empty do
4:    $x \leftarrow \text{pop}(\text{stack})$ 
5:   for each transition  $(x, \tau, y) \in \rightarrow$  do
6:     if  $y \notin \text{visited}$  then
7:       visited  $\leftarrow \text{visited} \cup \{y\}$ 
8:        $\text{push}(\text{stack}, y)$ 
9:     end if
10:  end for
11: end while
12: return visited
```

الگوریتم ۲.۳ یک برچسب غیر از τ را میان دو بستر داخلی قرار می‌دهد. اگر ℓ یک کنش قابل مشاهده باشد، خروجی همان $\tau^* \ell \tau^*$ است؛ اگر ℓ یک برچسب زمانی باشد، خروجی مقصدهای گذار ضعیف با همان مدت را می‌دهد. در حالت *unit*، تکرار این رابطه یک واحدی مدت‌های بزرگ‌تر را می‌سازد.

الگوریتم ۲.۳ Weak Successors for One Label

Require: A transition system T , tau closures C_τ , a state s , and a label $\ell \neq \tau$

Ensure: The targets of the weak step $s \xRightarrow{\ell}_W t$

```

1:  $targets \leftarrow \emptyset$ 
2: for each state  $x \in C_\tau(s)$  do
3:   for each transition  $(x, \ell, y) \in \rightarrow$  do
4:     for each state  $t \in C_\tau(y)$  do
5:        $targets \leftarrow targets \cup \{t\}$ 
6:     end for
7:   end for
8: end for
9: return  $targets$ 

```

الگوریتم ۳.۳ دو محاسبه قبلی را برای همه حالت‌ها و برچسب‌ها انجام می‌دهد. خروجی آن علاوه بر W ، بستارهای داخلی را نیز نگه می‌دارد؛ زیرا $C_\tau(s)$ حالت مدت صفر تعریف ۱.۸.۲ است و در پالایش افراز باید مانند دیگر حرکت‌های ضعیف بررسی شود.

الگوریتم ۴.۳ با قرار دادن همه حالت‌ها در یک بلوک آغاز می‌شود. در هر دور، برای هر حالت مجموعه بلوک‌های قابل دسترس با هر برچسب و مجموعه بلوک‌های موجود در بستار داخلی محاسبه می‌شود. دو حالت تنها زمانی در یک بلوک می‌مانند که بلوک فعلی و همه این مجموعه‌ها برای آن‌ها یکسان باشند. مقایسه آرایه $nextBlock$ با $block$ شرط توقف را به صورت صریح نشان می‌دهد.

پیاده‌سازی مبتنی بر امضا در هر دور، برای هر حالت و هر برچسب مجموعه بلوک‌های مقصد را بررسی می‌کند و حداکثر به تعداد حالت‌ها دور دارد. الگوریتم‌های پالایش پیشرفته‌تری با کران مجانبی بهتر وجود دارند، اما روش حاضر با شبه‌کد پروژه تطابق مستقیم دارد و برای بزرگ‌ترین ورودی موجود، که ۴۲ حالت پیش از پالایش زمانی دارد، کافی بوده است. فصل ۴ از این اندازه فراتر ادعای مقیاس‌پذیری نمی‌کند.

الگوریتم ۳.۳ Weak Transition Relation

Require: A timed transition system $T = (S, Act \cup \{\tau\} \cup D, \rightarrow)$

Ensure: Tau closures C_τ , non-tau labels L , and weak relation W

```

1: for each state  $s \in S$  do
2:    $C_\tau(s) \leftarrow \text{TauClosure}(T, s)$ 
3: end for
4:  $L \leftarrow \{\ell \mid (x, \ell, y) \in \rightarrow \wedge \ell \neq \tau\}$ 
5:  $W \leftarrow \emptyset$ 
6: for each state  $s \in S$  do
7:   for each label  $\ell \in L$  do
8:      $A \leftarrow \text{WeakSuccessors}(T, C_\tau, s, \ell)$ 
9:     for each state  $t \in A$  do
10:       $W \leftarrow W \cup \{(s, \ell, t)\}$ 
11:     end for
12:   end for
13: end for
14: return  $(C_\tau, L, W)$ 

```

 الگوریتم ۴.۳ Stable Partition Refinement

Require: States S , tau closures C_τ , labels L , and weak relation W

Ensure: The coarsest stable partition P

```

1: for each state  $s \in S$  do
2:    $block[s] \leftarrow 0$ 
3: end for
4: repeat
5:    $signatureIds \leftarrow$  an empty ordered map
6:   for each state  $s \in S$  do
7:     for each label  $\ell \in L$  do
8:        $R_\ell(s) \leftarrow \{block[t] \mid (s, \ell, t) \in W\}$ 
9:     end for
10:     $R_{\tau^*}(s) \leftarrow \{block[t] \mid t \in C_\tau(s)\}$ 
11:     $\sigma(s) \leftarrow (block[s], (R_\ell(s))_{\ell \in L}, R_{\tau^*}(s))$ 
12:    if  $\sigma(s) \notin signatureIds$  then
13:      Assign the next unused block id to  $signatureIds[\sigma(s)]$ .
14:    end if
15:     $nextBlock[s] \leftarrow signatureIds[\sigma(s)]$ 
16:  end for
17:   $changed \leftarrow (nextBlock \neq block)$ 
18:   $block \leftarrow nextBlock$ 
19: until  $changed = false$ 
20:  $P \leftarrow \{\{s \in S \mid block[s] = i\} \mid i \text{ is an assigned id}\}$ 
21: return  $P$ 

```

۱۰.۳ ساخت مدل خارج‌قسمتی و فشرده‌سازی زمان

در ساخت خارج‌قسمت، هر رده هم‌ارزی به یک حالت تبدیل می‌شود و اگر یالی با برچسب ℓ از عضوی از رده B به عضوی از رده B' وجود داشته باشد، یال $B' \xrightarrow{\ell} B$ ساخته می‌شود. خودحلقه τ درون یک رده حذف می‌شود؛ زیرا چیزی به بستار داخلی آن رده اضافه نمی‌کند.

خارج‌قسمت مستقیم سامانه پالایش شده فقط یال‌های یک‌واحدی دارد. در نتیجه، یک انتظار ده‌واحدی می‌تواند چندین رده میانی ایجاد کند و حتی مدل ظاهراً کاهش یافته را از ورودی بزرگ‌تر سازد. برای جلوگیری از این مسئله، زنجیره پیشینه‌ای از رده‌ها که تنها یک یال زمانی ورودی و یک یال زمانی خروجی دارند، حالت آغازین نیستند و هیچ انتخاب یا کنش قابل مشاهده‌ای ندارند، به یک یال با مدت مجموع تبدیل می‌شود. شرط ساختاری سخت‌گیرانه این مرحله مانع حذف نقطه‌ای می‌شود که محل انتخاب رفتاری است.

به همین دلیل، تعداد رده‌های افزاز لزوماً با تعداد حالت‌های خروجی یکسان نیست. هر رده یا به یک حالت خارج‌قسمتی تبدیل می‌شود یا موقعیت آن روی یک یال تأخیر فشرده‌شده ثبت می‌شود. فایل `partition.json` این نگاشت را نگه می‌دارد. همچنین، اطلاعات لازم برای بازکردن دوباره یال‌های چندواحدی ذخیره می‌شود تا واریسی‌گر بتواند دقیقاً مدل تحویلی را با سامانه پالایش شده مقایسه کند.

۱۱.۳ واریسی مستقل خروجی

واریسی‌گر رابطه میان حالت‌های سامانه پالایش شده و مدل خارج‌قسمتی را مستقیماً در برابر تعریف شبیه‌سازی دوسویه ضعیف زمان‌دار می‌سنجد. برای هر زوج مرتبط، همه حرکت‌های تهی و همه گذارهای ضعیف یک طرف باید با گذار هم‌برچسب طرف دیگر پاسخ داده شوند و مقصدها نیز مرتبط بمانند؛ این بررسی در هر دو جهت انجام می‌شود. حالت‌های آغازین نیز باید در رابطه باشند.

واریسی‌گر از بستارهای محاسبه‌شده در کاهش دهنده استفاده نمی‌کند. برای نمونه، بستارها را با مجموعه‌های معمولی از نو می‌سازد؛ درحالی‌که کاهش دهنده مقصدها را در ساختار BitSet نگه می‌دارد. استقلال کامل منطقی ادعا نمی‌شود، زیرا هر دو مؤلفه تعریف معنایی مشترکی دارند. بااین‌حال، جدایی کد باعث می‌شود خطای جزئی در شاخص‌گذاری یا عملیات بیتی کاهش دهنده، خودبه‌خود در واریسی‌گر تکرار نشود.

برای نشان‌دادن توان ردکردن، پنج تغییر مخرب روی مدل خارج‌قسمتی اعمال شده است: حذف یال، تغییر

برچسب، تغییر مدت، تغییر مقصد و افزودن یال. انتظار می‌رود واریسی‌گر همه این تغییرها را رد کند.

۱۲.۳ معماری نرم‌افزار و رابط اجرا

پیاده‌سازی با جاوای ۱۷ و Maven ساخته شده است. نسخه جاوا و نسخه‌های افزونه‌های ساخت با مخازن رسمی کامپایلر و واریسی‌گر مدل ربکا هم‌راستا شده‌اند [۱۶، ۱۸]. وابستگی زمان اجرا خارج از JDK وجود ندارد؛ خواندن فایل با StAX و تولید JSON با نویسنده‌ای کوچک و قطعی انجام می‌شود. این انتخاب احتمال تعارض کتاب‌خانه‌ها را در صورت ادغام آینده با محیط مبتنی بر Eclipse افرا کاهش می‌دهد. بسته‌های اصلی نرم‌افزار مسئولیت‌های زیر را دارند:

- afra و source: خواندن فایل و تبدیل منبع داده به نمایش خام؛
- tts: نمایش تغییرناپذیر و اعتبارسنجی شده سامانه انتقال؛
- weak: پالایش زمانی، ساخت گذارهای ضعیف و پالایش افراز؛
- quotient: ساخت خارج‌قسمت و فشرده‌سازی زنجیره‌های زمانی؛
- verify: بررسی مستقل رابطه خروجی؛
- report: تولید مصنوعات و سنجه‌ها؛
- app: هماهنگ‌کردن مراحل و رابط خط فرمان.

رابط خط فرمان^۴ چهار دستور دارد: inspect برای شناخت محتوای ورودی، reduce برای کاهش یک سامانه، equivalent برای مقایسه حالت آغازین دو سامانه، و visualize برای تولید نمایش Graphviz. منطق اصلی در ReductionService قرار دارد و خط فرمان تنها گزینه‌ها و کد خروج را مدیریت می‌کند؛ بنابراین، همان جریان بدون ایجاد فرایند جداگانه از کد جاوا نیز قابل فراخوانی است. کد خروج صفر موفقیت، یک نتیجه معنایی منفی، دو ورودی نامعتبر و سه نقض یک ناوردای داخلی را نشان می‌دهد.

^۴Command-Line Interface

۱۳.۳ مصنوعات خروجی

جدول ۲.۳ نقش پنج فایل تولیدشده برای هر کاهش را نشان می‌دهد. هیچ‌یک از این فایل‌ها به‌تنهایی جای همه بقیه را نمی‌گیرد؛ برای نمونه، نمایش سازگار با افرا برای مشاهده مناسب است، ولی عضویت رده‌ها و منشأ کامل یال‌ها را نگه نمی‌دارد.

جدول ۲.۳: مصنوعات تولیدشده برای هر اجرای کاهش

فایل	محتوا و کاربرد
reduced.json	نمایش اصلی و بدون اتلاف مدل کاهش‌یافته، شامل برچسب‌ها، اعضای رده‌ها و مجموعه مشاهده‌ها.
partition.json	عضویت هر حالت ورودی در رده و موقعیت آن در مدل کاهش‌یافته.
reduced.statespace	نمای سازگار با ابزار نمایش فضای حالت افرا؛ این نما به‌دلیل محدودیت طرح داده اتلاف‌دار است.
reduced.dot	گراف DOT برای ترسیم ساختار مدل خارج‌قسمتی.
metrics.json	اندازه‌ها، نسبت‌های کاهش، زمان و حافظه، نتیجه واری و اطلاعات ردیابی اجرا.

قطعی‌بودن ترتیب حالت‌ها، رده‌ها و یال‌ها موجب می‌شود چهار مصنوع ساختاری در دو اجرای یکسان، بابت به‌بایت برابر باشند. زمان اجرا و بیشینه حافظه طبیعتاً در metrics.json متغیرند. هش SHA-256 ورودی و شناسه کامیت امکان اتصال هر نتیجه به فایل و نسخه مولد آن را فراهم می‌کند.

۱۴.۳ طرح اعتبارسنجی و تکرار اجرا

اعتبارسنجی در سه سطح انجام شده است. در سطح معنایی، نمونه‌های کوچک با افراز مورد انتظار، بستر T ، مشاهده‌ها، تأخیرها، بن‌بست و ورودی نامعتبر را می‌سنجند. در سطح مدل خارج‌قسمتی، کامل‌بودن افراز، معتبربودن مقصد یال‌ها، رابطه حالت آغازین و رد جهش‌های عمده بررسی می‌شود. در سطح انتها به انتها، فایل statespace به برنامه بسته‌بندی شده داده می‌شود و کد خروج، شمار حالت‌ها و محتوای همه مصنوعات کنترل می‌شوند. جزئیات عددی این آزمون‌ها در فصل ۴ آمده است.

برای تکرار ساخت و آزمون نسخه ثبت‌شده پروژه، فرمان‌های اصلی به‌شکل زیرند:

```
mvn clean package
python3 tests/e2e/run_e2e.py
python3 evaluation/run_evaluation.py
```

فرمان نخست نرم‌افزار و آزمون‌های واحد را می‌سازد، فرمان دوم رابط بسته‌بندی شده را مانند یک کاربر خارجی می‌آزماید و فرمان سوم فایل جمع‌بندی و سنجه‌های خام ارزیابی را از نو تولید می‌کند. ورودی‌ها، اسکریپت‌ها و نتایج ثبت شده در مخزن پروژه موجودند [۱۲].

۱۵.۳ جمع‌بندی

روش این پروژه یک زنجیره قابل ردیابی از خروجی TTS افرا تا مدل خارج‌قسمتی است. مجموعه مشاهده‌ها صریح است، تفاوت دو تفسیر تأخیر به گزینه و آزمون تبدیل شده، پالایش افراز از قرارداد ورودی جدا مانده و مدل تحویلی با پیاده‌سازی دیگری از شروط انتقال بررسی می‌شود. فصل بعد نشان می‌دهد این تصمیم‌ها روی نمونه‌های ارزیابی چه خروجی‌ای تولید کرده‌اند و شواهد تا چه اندازه از درستی و کارایی روش پشتیبانی می‌کنند.

فصل ۴

نتایج و اعتبارسنجی

۱.۴ مقدمه

این فصل خروجی ابزار را روی چهار سامانه انتقال گزارش می‌کند و سپس شواهد اعتبارسنجی را بررسی می‌کند. پرسش اصلی این است که آیا روش فصل ۳ می‌تواند با حفظ رفتار ضعیف زمان‌دار، مدل کوچک‌تری تولید کند و آیا نمونه‌هایی که رفتار یکسان را با تقسیم‌بندی متفاوت زمان بیان می‌کنند به نتیجه‌ای سازگار می‌رسند. نتیجه‌ها از فایل‌های خام تولیدشده توسط اسکریپت ارزیابی گرفته شده‌اند؛ بنابراین، شمار حالت‌ها، گذارها و سنجه‌های اجرا به یک ورودی و نسخه مشخص از برنامه قابل ردیابی اند [۱۲].

این ارزیابی برای استنتاج آماری یا تعیین روند رشد زمان اجرا طراحی نشده است. تعداد مدل‌ها کم است و هر اندازه‌گیری زمان تنها یک اجرای ثبت‌شده را نشان می‌دهد. از اعداد کارایی برای توصیف هزینه نمونه‌های موجود استفاده می‌شود، نه برای ادعای مقیاس‌پذیری روی مدل‌های صنعتی.

۲.۴ ورودی‌های ارزیابی

جدول ۱.۴ چهار ورودی و مجموعه مشاهده‌های آن‌ها را نشان می‌دهد. نمونه tiny یک سامانه کوچک و قابل بررسی دستی است که دو مسیر با مدت کل برابر دارد. سه نمونه دیگر از نمودارهای مطالعه موردی خانه هوشمند

استخراج شده‌اند. در هر سه، تنها پیام‌های `getSense`، `activateh` و `switchoff` قابل مشاهده‌اند و همه پیام‌های دیگر به T تبدیل می‌شوند.

جدول ۱.۴: ورودی‌ها و هدف هر مورد ارزیابی

مورد	تعامل‌های قابل مشاهده	هدف
tiny	poll	بررسی دستی ادغام دو مسیر با مدت کل یکسان.
smart-home	getSense, activateh, switchoff	نمونه پایه خانه هوشمند با یک تأخیر ده‌واحدی.
smart-home-tc2step	همان سه تعامل	بیان همان دوره با تأخیرهای هفت و سه‌واحدی و یک گام داخلی میان آنها.
smart-home-notify	همان سه تعامل	نمونه بزرگ‌تر با مؤلفه اعلان و تقسیم دوره به سه و هفت واحد.

سه فایل خانه هوشمند از روی نمودارهای ارائه‌شده برای پروژه به جدول یال تبدیل شده‌اند. شمار حالت‌ها و یال‌ها، جهت یال‌ها، حالت آغازین و قواعد یکسان‌سازی نام‌ها در مستند منشأ داده ثبت شده است. فایل سازگار با قالب افرا نیز از روی نمایش Graphviz رسمی بازسازی شده، نه اینکه مستقیماً از اجرای افرا کپی شده باشد. این تفاوت بر ساختار یال‌های مورد ارزیابی اثر شناخته‌شده‌ای ندارد، اما منشأ بازسازی‌شده داده دامنۀ اعتبار خارجی نتیجه را محدود می‌کند.

۳.۴ شرایط اجرا و سنجه‌ها

نتایج ثبت‌شده با نسخه 1.0.0 ابزار و کامیت 3d63904 تولید شده‌اند. محیط اجرا Darwin 23.6.0 روی معماری arm64 با ۸ پردازنده منطقی و OpenJDK 17.0.10 بوده است. اسکرپت ارزیابی هر ورودی را در یک فرایند جدا اجرا کرده و فایل metrics.json همان اجرا را در کنار خروجی‌های ساختاری ذخیره کرده است [۱۲].

اگر $|S_o|$ و $|E_o|$ به ترتیب تعداد حالت‌ها و گذارهای ورودی و $|S_r|$ و $|E_r|$ مقادیر مدل کاهش‌یافته باشند،

نسبت‌های گزارش شده از رابطه‌های زیر به دست می‌آیند:

$$\rho_S = 1 - \frac{|S_r|}{|S_o|}, \quad \rho_E = 1 - \frac{|E_r|}{|E_o|}.$$

ستون «پس از پالایش زمانی» اندازه نمایش میانی بعد از تبدیل تأخیرهای چندواحدی به یال‌های واحد است و نباید با اندازه ورودی اشتباه گرفته شود. «تعداد رده‌ها» نیز روی همین نمایش پالایش شده محاسبه می‌شود. همان‌طور که در فصل قبل توضیح داده شد، برخی رده‌های صرفاً زمانی هنگام ساخت خروجی روی یک یال فشرده می‌شوند؛ از این‌رو، تعداد رده‌ها ممکن است از تعداد حالت‌های مدل نهایی بیشتر باشد.

۴.۴ نتایج کاهش

جدول ۲.۴ اندازه‌های اصلی را نشان می‌دهد. واریسی‌گر مستقل، رابطه مدل ورودی و مدل کاهش یافته را برای هر چهار مورد پذیرفته است. این پذیرش به معنای مینیمال بودن مدل خروجی نیست؛ نتیجه پشتیبانی شده این است که خروجی، مطابق معنای زمان گسسته انتخاب شده، با ورودی شبیه‌سازی دوسویه ضعیف زمان‌دار دارد.

جدول ۲.۴: شمار حالت‌ها، گذارها و نتیجه کاهش

مورد	ورودی	پس از پالایش زمانی	رده‌ها	خروجی
tiny	۶ حالت و ۷ گذار	11 / 12	۵	2 / 2
smart-home	16 / 18	34 / 36	۲۸	10 / 12
smart-home-tc2step	25 / 28	41 / 44	۲۸	10 / 12
smart-home-notify	42 / 52	85 / 95	۲۸	10 / 12

در نمونه کوچک، شش حالت و هفت گذار به دو حالت و دو گذار کاهش یافته‌اند. در سه نمونه خانه هوشمند، با وجود تفاوت اندازه ورودی‌ها، پالایش افراز در هر مورد ۲۸ رده ساخته و مرحله فشرده‌سازی زمانی به مدل‌هایی با ۱۰ حالت و ۱۲ گذار رسیده است. بیشترین نسبت کاهش میان ورودی‌های موجود به smart-home-notify 76.2 درصد از حالت‌ها و 76.9 درصد از گذارها حذف شده‌اند. این عدد فقط برای مجموعه مشاهده‌ی سه‌عضوی جدول ۱.۴ معتبر است؛ انتخاب مشاهده‌های بیشتر یا کمتر می‌تواند افراز دیگری بسازد.

جدول ۳.۴: نسبت کاهش حالت‌ها و گذارها

مورد	کاهش حالت‌ها	کاهش گذارها
tiny	66.7 درصد	71.4 درصد
smart-home	37.5 درصد	33.3 درصد
smart-home-tc2step	60.0 درصد	57.1 درصد
smart-home-notify	76.2 درصد	76.9 درصد

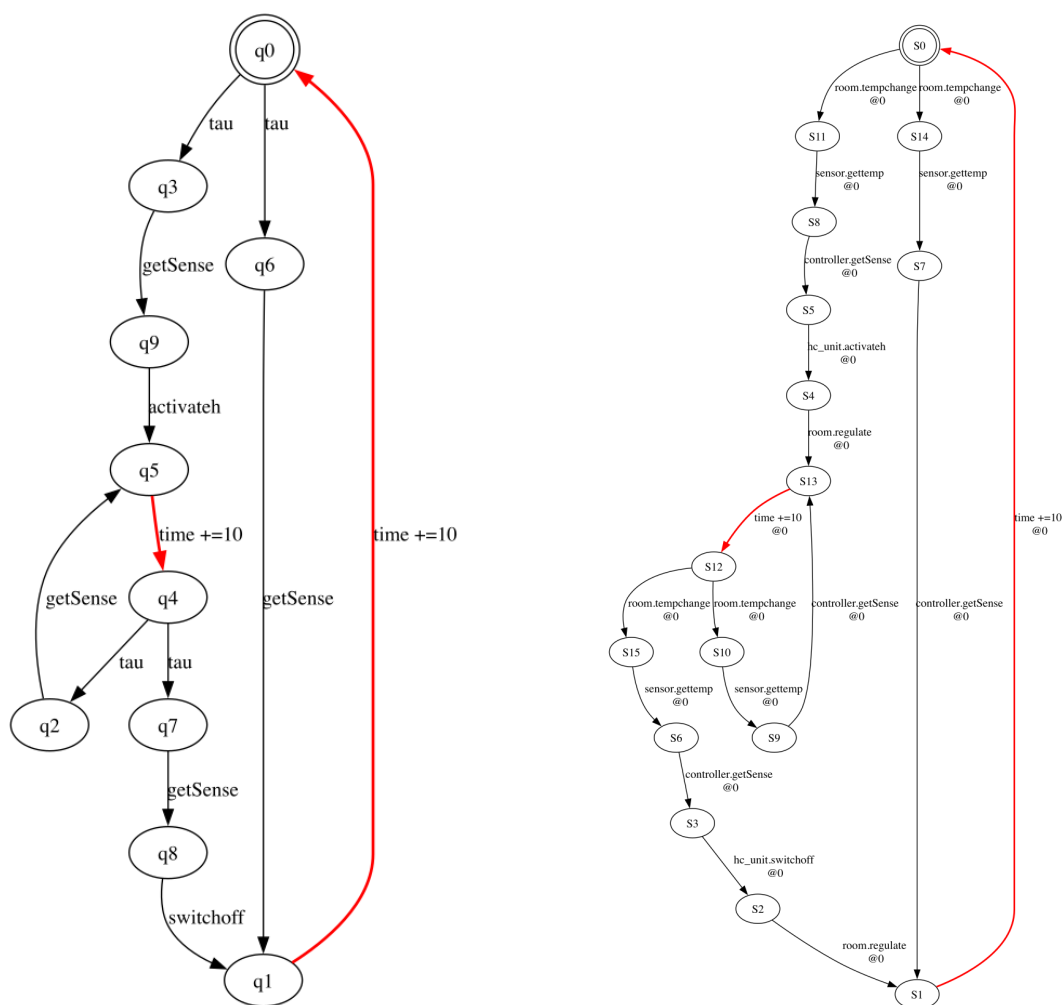
۵.۴ مطالعه موردی خانه هوشمند

شکل ۱.۴ فضای حالت پایه و خارج قسمت آن را کنار هم قرار می‌دهد. یال‌های قرمز گذر زمان را نشان می‌دهند. نمودار نخست ۱۶ حالت و ۱۸ گذار دارد و نمودار دوم همان رفتار قابل مشاهده را با ۱۰ حالت و ۱۲ گذار نمایش می‌دهد. هر دو تصویر با ابزارهای همان مخزن از ورودی smart-home.statespace تولید شده‌اند [۱۲].

سه ورودی خانه هوشمند یک رفتار را با جزئیات داخلی متفاوت بیان می‌کنند. بررسی بازنمایی‌های اصلی خروجی نشان می‌دهد هر سه، صرف‌نظر از نام حالت‌ها، به ساختار خارج قسمتی یکسان رسیده‌اند. این نتیجه با هم‌ارزی جفت به جفت آن‌ها سازگار است. افزون بر پاسخ مثبت دستور equivalent، این ساختار مشترک شاهدهی برای همگرایی سه ورودی فراهم می‌کند.

با این حال، این مشاهده قضیه‌ای دربارهٔ یکتا بودن خارج قسمت برای همهٔ سامانه‌های هم‌ارز نیست؛ هم‌ارزی ضعیف در حالت کلی لزوماً خارج قسمت کانونی تولید نمی‌کند.

تفاوت ۲۸ ردهٔ جدول ۲.۴ با ۱۰ حالت خروجی ناشی از حذف رده‌های هم‌ارز نیست. پالایش واحدی، لحظه‌های میانی گذر زمان را به حالت تبدیل می‌کند و افراز نیز برای آن‌ها رده می‌سازد. مرحلهٔ فشرده‌سازی، زنجیره‌های بدون انتخاب را دوباره به یال‌های چندواحدی تبدیل می‌کند. نگاشت این رده‌ها در partition.json باقی می‌ماند؛ بنابراین، هر حالت ورودی یا به یک حالت خروجی و یا به نقطه‌ای مشخص روی یک یال زمان‌دار قابل ردیابی است.



(ب) مدل خارج‌قسمتی واری شده

(الف) فضای حالت ورودی

شکل ۱.۴: فضای حالت خانه هوشمند پیش و پس از کاهش ضعیف زمان‌دار.

۶.۴ حساسیت به معنای زمان و مجموعه مشاهده‌ها

جدول ۴.۴ نتیجه مقایسه نمونه پایه با نسخه tc2step را تحت دو تفسیر بخش ۶.۳ نشان می‌دهد. با جمع‌پذیری زمان، تأخیر ده‌واحدی می‌تواند پاسخ اجرای $3 + \tau + 7$ باشد و دو حالت آغازین در یک رده قرار می‌گیرند. در حالت strict، هر طرف باید یک یال منفرد با همان مدت داشته باشد و مقایسه رد می‌شود.

جدول ۴.۴: حساسیت نتیجه به تفسیر تأخیر

نتیجه	ملاک تطبیق تأخیر	حالت
هم‌ارز	اجرای بدون کنش قابل مشاهده با مدت کل برابر	unit
ناهم‌ارز	یک یال منفرد با برچسب مدت برابر	strict

این آزمون نشان می‌دهد انتخاب معنای زمان فقط یک تفاوت پیاده‌سازی نیست و مستقیماً حکم هم‌ارزی را تغییر می‌دهد. وجود هر دو حالت در رابط برنامه امکان‌پذیری فرض را حفظ می‌کند. نتیجه مثبت اصلی فصل بر حالت unit تکیه دارد و نباید به خوانش strict تعمیم داده شود.

آزمون انتهابه‌انتها همچنین بررسی می‌کند تغییر مجموعه مشاهده‌ها افراز را تغییر می‌دهد و همه اجراها همچنان خارج‌قسمتی پذیرفته‌شده تولید می‌کنند. این نتیجه مورد انتظار است: پنهان‌کردن تعامل‌های بیشتر، قدرت تمایز ناظر را کاهش می‌دهد و ممکن است حالت‌های بیشتری قابل ادغام شوند. در مقابل، یکسان‌سازی خودکار بزرگی و کوچکی حروف انجام نمی‌شود؛ مقایسه نام GETSENSE در یک خروجی با getSense در ورودی دیگر نتیجه منفی می‌دهد.

۷.۴ نتایج واریسی رابطه خروجی

جدول ۵.۴ تعداد زوج‌های رابطه و تعهدهای انتقال بررسی شده را از فایل‌های metrics.json گزارش می‌کند. منظور از تعهد انتقال، یک حرکت ضعیف در یکی از دو جهت است که باید پاسخ متناظر آن پیدا شود. در همه موارد، فهرست نقض‌ها تهی بوده است.

صرف پذیرفته‌شدن خروجی صحیح نشان نمی‌دهد واریسی‌گر توان کشف خطا دارد. برای کنترل این مسئله، پنج جهش روی خارج‌قسمت اعمال شده است: حذف یال، تغییر برچسب تعامل، تغییر مدت، تغییر مقصد

جدول ۵.۴: خلاصه واریسی مستقل مدل های کاهش یافته

مورد	زوج های مرتبط	تعهد های بررسی شده	نتیجه
tiny	۱۱	۵۲	پذیرفته
smart-home	۳۴	۱۷۲	پذیرفته
smart-home-tc2step	۴۱	۲۱۴	پذیرفته
smart-home-notify	۸۵	۴۵۴	پذیرفته

و افزودن یال. هر پنج مدل جهش یافته رد شده اند. یک آزمون جدا نیز با تغییر مدت در ورودی، حکم هم ارزی مثبت را به منفی تبدیل می کند. این نتیجه ها احتمال پذیرش همیشگی یا بررسی ناقص یک طرفه را کاهش می دهند، هر چند جای اثبات صوری خود برنامه را نمی گیرند.

۸.۴ پوشش آزمون ها

ساخت ثبت شده شامل ۱۰۶ آزمون مبتنی بر JUnit است که بدون شکست، خطا یا صرف نظر اجرا شده اند. توزیع آن ها در جدول ۶.۴ آمده است. افزون بر آن، اسکریپت انتهایی ۶۲ بررسی را از طریق فایل JAR بسته بندی شده اجرا کرده و همه آن ها پذیرفته شده اند.

جدول ۶.۴: تعداد آزمون ها بر حسب هدف

تعداد	گروه آزمون
۲۱	خط مبنای قدیمی در قالب آزمون جعبه سیاه
۱۲	نمونه های پذیرش خانه هوشمند و جهش های آن ها
۲۳	معنای گذار ضعیف، بستارها، تأخیرها و پایداری افزاز
۲۳	ساختار خارج قسمت، پذیرش رابطه و جهش های واریسی گر
۱۷	خواننده افرا و ورودی های نامعتبر
۸	مرز معماری و قرارداد آداپتور
۲	تولید نمایش Graphviz
۱۰۶	جمع آزمون های JUnit
۶۲	بررسی های انتهایی

آزمون انتهایی وجود و محتوای پنج مصنوع خروجی، کدهای خروج، ورودی نامعتبر و اثر مجموعه

مشاهده‌ها را بررسی می‌کند. حساسیت معنای زمان، جهش یک مدل و قطعی بودن فایلهای ساختاری نیز در همین سطح آزموده می‌شوند.

این آزمون فقط برنامه بسته‌بندی شده را از بیرون اجرا می‌کند. از این رو، موفقیت آن شواهدی جدا از فراخوانی مستقیم کلاس‌های داخلی فراهم می‌آورد [۱۲].

۹.۴ زمان اجرا و حافظه

جدول ۷.۴ زمان درون‌فرایندی گزارش شده توسط ابزار، زمان دیواری اندازه‌گیری شده توسط اسکریپت و بیشینه حافظه پشته جاوا را نشان می‌دهد. حافظه با واحد مگابایت دهدهی گزارش شده است.

جدول ۷.۴: زمان و بیشینه حافظه در اجرای ثبت شده

مورد	زمان درون‌فرایندی (میلی ثانیه)	زمان دیواری (میلی ثانیه)	حافظه (مگابایت)
tiny	۵۰	130.3	6.2
smart-home	۵۵	137.0	7.2
smart-home-tc2step	۵۸	139.5	7.2
smart-home-notify	۶۳	153.3	8.3

فاصله زمان درون‌فرایندی و زمان دیواری نشان می‌دهد هزینه راه‌اندازی ماشین مجازی جاوا سهم قابل توجهی از اجرای این نمونه‌های کوچک را تشکیل می‌دهد. اختلاف زمان‌ها کم است، اما چهار نقطه و یک اجرا برای هر مدل برای برازش روند یا مقایسه جانبی کافی نیست. نتیجه محتاطانه این است که همه نمونه‌های موجود با حافظه کمتر از ۹ مگابایت و زمان دیواری کمتر از ۱۶۰ میلی ثانیه در محیط ثبت شده پردازش شده‌اند؛ این مشاهده به ورودی‌های بزرگ‌تر قابل تعمیم نیست.

۱۰.۴ جمع‌بندی

هر چهار ورودی به مدل کوچک‌تری رسیده‌اند و واری‌گر مستقل رابطه خروجی آن‌ها را پذیرفته است. سه بیان متفاوت از خانه هوشمند، با مجموعه مشاهده یکسان و معنای جمع‌پذیر زمان، به خارج قسمت‌هایی

هم‌ساختار با ۱۰ حالت و ۱۲ گذار رسیده‌اند. آزمون حالت strict نشان داد این نتیجه به تصمیم معنایی فصل ۳ وابسته است.

مجموعه آزمون‌ها، جهش‌های منفی و مصنوعات قابل ردیابی از نتایج موجود پشتیبانی می‌کنند. با این حال، اندازه کوچک ورودی‌ها و منشأ بازسازی‌شده فایل افرا اجازه نمی‌دهد درباره مقیاس‌پذیری یا پوشش کامل زنجیره رسمی افرا ادعایی مطرح شود.

فصل ۵

بحث و نتیجه‌گیری

۱.۵ مرور مسیر پروژه

این پروژه از یک مسئله عملی در زنجیره تحلیل ربکا آغاز شد: فضای حالت زمان‌دار تولید شده توسط افرا می‌تواند جزئیاتی داشته باشد که برای سطح مشاهده انتخابی کاربر ضروری نیستند، اما تحلیل‌گر همچنان باید همه آن‌ها را نگهداری و پیمایش کند. تعریف شبیه‌سازی دوسویه ضعیف زمان‌دار مبنایی برای نادیده گرفتن کنش‌های داخلی با حفظ رفتار قابل مشاهده و زمان فراهم می‌کند [۲۲، ۱۰]. فاصله میان این تعریف و یک ابزار قابل استفاده شامل چند مسئله مهندسی و معنایی بود: قرارداد واقعی فایل افرا، هویت تعامل‌ها، تعبیر تأخیرهای چندمرحله‌ای، ساخت مدل خارج‌قسمتی و واریسی نتیجه تحویلی.

کار اصلی پروژه پیاده‌سازی این فرایند در ابزار awtr بود. ابزار فایل statespace را می‌خواند، تعامل‌های خارج از مجموعه مشاهده را پنهان می‌کند، بخش دسترس‌پذیر مدل را نگه می‌دارد، تأخیرها را مطابق معنای انتخابی پالایش می‌کند و پس از ساخت رابطه گذار ضعیف، افراز پایدار و مدل خارج‌قسمتی را به دست می‌آورد. واریسی جدایانه رابطه مدل پالایش شده و خروجی را در هر دو جهت بررسی می‌کند. نتیجه همراه با نداشت افراز، منشأ یال‌ها و سنجه‌های اجرا ذخیره می‌شود تا صرفاً به یک پاسخ مثبت بدون شاهد اکتفا نشود.

۲.۵ پاسخ به پرسش‌های پروژه

۱.۲.۵ خواندن و مستقل کردن خروجی افرا

پرسش نخست درباره امکان مصرف مستقیم خروجی زمان‌دار افرا بود. بررسی تولیدکننده و خواننده‌های رسمی نشان داد فایل واقعی TTS قراردادی دارد که تنها از روی طرح داده روشن نمی‌شود؛ از جمله، عنصر ریشه ممکن است در انتهای فایل بسته نشده باشد و نخستین حالت نقش آغازین را داشته باشد. خواننده پیاده‌سازی شده این موارد را بدون تغییر دستی فایل مدیریت و داده را پشت رابطی مستقل از افرا به مدل دامنه تبدیل می‌کند. آزمون قرارداد نشان می‌دهد منبع فایل و منبع درون حافظه‌ای به نتیجه یکسان می‌رسند. بنابراین، برای دامنه TTS و فیلدهای مستندشده، پاسخ پرسش نخست مثبت است. این نتیجه به FTTS تعمیم داده نمی‌شود.

۲.۲.۵ کاهش همراه با واریسی مستقل

پرسش دوم ساخت یک مدل کوچک‌تر و واریسی رابطه آن با ورودی بود. هر چهار ورودی ارزیابی به خروجی کوچک‌تری رسیدند و واریسی گر مستقل همه رابطه‌های تحویلی را پذیرفت. برای سنجش اینکه پذیرش صرفاً نتیجه یک واریسی گر همیشه مثبت نیست، پنج نوع تغییر مخرب روی مدل خارج‌قسمتی اعمال شد و واریسی گر همه را رد کرد. ۱۰۶ آزمون JUnit و ۶۲ بررسی انتهابه‌انتها نیز بدون شکست اجرا شدند. این شواهد از درستی خروجی‌های موجود پشتیبانی می‌کنند، اما جای اثبات درستی خود پیاده‌سازی یا بازبینی مستقل تعریف را نمی‌گیرند.

واژه «کاهش» در این نتیجه باید دقیق تفسیر شود. مدل خروجی رفتاری را که مجموعه مشاهده و معنای زمان تعیین می‌کند حفظ کرده است؛ با تغییر مجموعه مشاهده، افراز نیز می‌تواند تغییر کند. همچنین، ابزار کوچک‌ترین مدل ممکن یا یک نمایش کانونی را تضمین نمی‌کند. ادعای پشتیبانی شده این است که خارج‌قسمت‌های تولیدشده برای نمونه‌های ارزیابی از ورودی کوچک‌تر و مطابق واریسی گر، با آن دارای شبیه‌سازی دوسویه ضعیف زمان‌دار بوده‌اند.

۳.۲.۵ اثر معنای زمان و اندازه کاهش

پرسش سوم به تصمیم معنایی و مقدار کاهش مربوط بود. در حالت پیش فرض unit، تأخیر چندواحدی به گام های واحد شکسته می شود؛ بنابراین یک تأخیر ده واحدی می تواند با اجرای $3 + \tau + 7$ یا $7 + \tau + 3$ تطبیق یابد. سه بیان مطالعه موردی خانه هوشمند در این معنا هم ارز شناخته شدند و پس از فشرده سازی به ساختاری با ۱۰ حالت و ۱۲ گذار رسیدند. در حالت strict، که فقط یک یال منفرد هم مدت را می پذیرد، نمونه پایه و tc2step هم ارز نبودند. پس نتیجه هم ارزی مستقیماً به تعبیر جمع پذیری زمان وابسته است و این تعبیر باید جزئی از قرارداد تحلیل باقی بماند، نه یک جزئیات پنهان پیاده سازی.

در ورودی کوچک، تعداد حالت ها از ۶ به ۲ و تعداد گذارها از ۷ به ۲ کاهش یافت. برای سه نمونه خانه هوشمند، نسبت کاهش حالت به ترتیب 37.5، 60.0 و 76.2 درصد و نسبت کاهش گذار 33.3، 57.1 و 76.9 درصد بود. افزایش نسبت کاهش در نمونه های بزرگ تر این مجموعه با افزوده شدن جزئیات داخلی سازگار است، اما چهار نقطه ارزیابی برای نتیجه گیری درباره روند مقیاس پذیری کافی نیست.

۳.۵ تفسیر تصمیم های طراحی

پالایش تأخیرها به گام های واحد، تعریف جمع پذیر زمان را به مسئله ای متناهی با یک برچسب زمانی واحد تبدیل می کند. مزیت آن سادگی و امکان استفاده از همان پالایش افراز برای کنش و زمان است. هزینه آن افزودن $1 - d$ حالت برای یک یال با مدت d است. فشرده سازی زنجیره های صرفاً زمانی پس از ساخت خارج قسمت، اندازه مدل تحویلی را کاهش می دهد، ولی هزینه نمایش میانی را حذف نمی کند. بنابراین این تصمیم برای ورودی های فعلی مناسب بوده است، نه لزوماً برای مدل هایی با ثابت های زمانی بسیار بزرگ.

جداسازی واریسی گر از کاهش دهنده نیز یک مصالحه مهندسی است. واریسی گر بستارها و تعهد های رابطه را با ساختار داده و مسیر کد متفاوتی محاسبه می کند؛ از این رو، بخشی از خطاهای شاخص گذاری، ساخت امضا یا فشرده سازی می تواند آشکار شود. با این حال، هر دو مؤلفه تعبیر معنایی مشترکی دارند و خطای مشترک در همان تعبیر ممکن است از هر دو عبور کند. آزمون جهش اعتماد به توان رد کردن خروجی معیوب را افزایش می دهد، اما اثبات صوری واریسی گر نیست.

خروجی های چندگانه نیز دو نیاز متفاوت را پاسخ می دهند. reduced.statespace برای مشاهده در ابزارهای

افرا سودمند است، ولی قالب آن همه منشأها و عضویت رده‌ها را نگه نمی‌دارد. در مقابل، `reduced.json` نتیجه اصلی و بدون اتلاف پروژه است و `partition.json` امکان ردیابی حالت ورودی تا حالت یا یال خروجی را فراهم می‌کند. این تفکیک اجازه می‌دهد سازگاری با ابزار موجود، اطلاعات لازم برای بازتولید نتیجه را محدود نکند.

۴.۵ جایگاه دستاورد

این پایان‌نامه تعریف تازه‌ای از شبیه‌سازی دوسویه یا روش نظری تازه‌ای برای حل آن ارائه نمی‌کند. تعریف رفتاری و ایده تطبیق زمان بر پژوهش‌های پیشین استوارند [۲۲، ۱۰]. سهم پروژه در سطح کارشناسی، تبدیل این مبنا به یک مسیر اجرایی و آزموده برای مدل معنایی ربکا است: قرارداد ورودی افرا از کدهای رسمی استخراج شده، ابهام معنای تأخیر به دو حالت قابل انتخاب تبدیل شده، مدل خارج‌قسمتی همراه با منشأ ساخته شده و نتیجه با پیاده‌سازی جداگانه و آزمون‌های منفی بررسی شده است.

از دید کاربردی، ابزار موجود یک نمونه مستقل از خط فرمان است و هنوز جزئی از محیط افرا نیست. در نتیجه، دستاورد فعلی را باید زیرساختی برای ارزیابی روش و ادغام آینده دانست، نه بهبود اثبات‌شده کارایی خود افرا. داده‌های فصل ۴ نشان می‌دهند کاهش روی نمونه‌های موجود رخ داده و هزینه اجرا در محیط ثبت شده کم بوده است؛ اما چون زمان تحلیل بعدی با افرا اندازه‌گیری نشده، نمی‌توان از این نتایج درباره شتاب نهایی واریسی مدل ادعای کمی مطرح کرد.

۵.۵ محدودیت‌ها

محدودیت‌های اصلی نتیجه عبارت‌اند از:

- بزرگ‌ترین ورودی پیش از پالایش زمانی ۴۲ حالت دارد و نماینده یک مدل صنعتی نیست؛
- ارزیابی فقط چهار ورودی و یک خانواده کاربردی را پوشش می‌دهد و هر زمان اجرا یک‌بار ثبت شده است؛

- فایل افرا از روی نمایش تولید شده بازسازی شده و خروجی مستقیم یک اجرای رسمی در دسترس نبوده است؛
 - پشتیبانی زمانی به TTS، زمان گسسته و تأخیرهای صحیح محدود است؛
 - پالایش واحدی می تواند برای تأخیرهای بزرگ نمایش میانی بسیار بزرگی بسازد؛
 - نتیجه های هم ارزی به قرارداد معنایی تأیید شده جمع پذیری زمان وابسته اند و به خوانش strict تعمیم داده نمی شوند؛
 - خارج قسمت واری می شود، اما مینیمال یا کانونی بودن آن اثبات نشده است؛
 - سنجش حاضر کاهش فضای حالت را گزارش می کند و اثر آن بر زمان یک واری مدل کامل را اندازه نمی گیرد.
- این محدودیت ها نتیجه اصلی روی نمونه های ثبت شده را نقض نمی کنند، اما مرز تعمیم آن را تعیین می کنند. به ویژه، درصد های کاهش تنها همراه با مجموعه مشاهده و معنای زمانی همان اجرا قابل تفسیرند.

۶.۵ کارهای آینده

نخستین ادامه طبیعی، افزودن آداپتور درون فرایندی و فرمان متناظر در افراست تا مدل بدون فایل واسط به هسته کاهش تحویل شود. پیش از آن، نمونه بازسازی شده باید با یک خروجی مستقیم و قابل انتشار افرا جایگزین و قرارداد خواننده روی چند مدل واقعی دیگر بررسی شود. تشخیص صریح FTTS نیز باید افزوده شود تا ابزار به جای تفسیر خاموش یک قالب پشتیبانی نشده، خطای روشن ارائه کند.

در سطح معنایی، جمع پذیری زمان به عنوان قرارداد اصلی روش تثبیت شده است. مقایسه تفاضلی با یک پیاده سازی مستقل دیگر یا صورت بندی مرجع می تواند خطر خطای مشترک میان کاهش دهنده و واری گر را کمتر کند. اثبات برخی ناوردهای کلیدی، مانند حفظ رابطه در فشرده سازی زنجیره زمان، گام بعدی دقیق تری از افزودن آزمون های مشابه است.

برای مقیاس بزرگ‌تر، می‌توان از پالایش افراز کارآمدتر و نمایش نمادین زمان با ناحیه یا زون استفاده کرد تا نیاز به ساخت یک حالت برای هر واحد تأخیر کاهش یابد. ارزیابی این نسخه باید مجموعه‌ای متنوع از مدل‌های ربکا، تأخیرهای بزرگ‌تر و تعداد اجرای تکراری داشته باشد و علاوه بر اندازه خارج‌قسمت، زمان و حافظه کل جریان واریسی مدل را پیش و پس از کاهش مقایسه کند.

در نهایت، قالب خروجی افرا ظرفیت نگهداری همه منشأهای یک یال خارج‌قسمتی را ندارد. تعریف یک افزونه سازگار برای این فراداده یا نمایش مستقیم آن در رابط افرا می‌تواند قابلیت ردیابی JSON را به جریان تصویری منتقل کند و بررسی دستی علت ادغام حالت‌ها را آسان‌تر سازد.

۷.۵ جمع‌بندی نهایی

نتایج نشان می‌دهند تعریف شبیه‌سازی دوسویه ضعیف زمان‌دار را می‌توان برای خروجی TTS افرا به یک فرایند کاهش قابل اجرا و قابل ردیابی تبدیل کرد. ابزار ساخته‌شده در هر چهار ورودی ارزیابی مدلی کوچک‌تر تولید کرد، رابطه خروجی‌ها در واریسی مستقل پذیرفته شد و آزمون‌های منفی توانستند تغییرهای معیوب را آشکار کنند. مطالعه موردی همچنین نشان داد یک تصمیم ظاهراً کوچک درباره ترکیب تأخیرها می‌تواند پاسخ هم‌ارزی را تغییر دهد؛ بنابراین، ثبت معنای زمان به اندازه خود الگوریتم اهمیت دارد.

این نتیجه یک گام مهندسی برای پیوند مبانی نظری و زنجیره ابزار ربکا و افراست. شواهد موجود برای درستی نمونه‌های بررسی شده و امکان کاهش آن‌ها کافی‌اند، اما برای ادعای مقیاس‌پذیری، مینیمال‌بودن خروجی یا بهبود کارایی افرا کافی نیستند. ادامه کار باید بر تثبیت معنای زمان، ورودی‌های واقعی‌تر، ادغام با افرا و ارزیابی در مقیاس بزرگ‌تر متمرکز شود.

کتاب نامه

- [1] Alur, Rajeev and Dill, David L. Automata for modeling real-time systems. In *Automata, Languages and Programming*, volume 443 of *Lecture Notes in Computer Science*, pages 322–335. Springer, 1990.
- [2] Alur, Rajeev and Dill, David L. A theory of timed automata. *Theoretical Computer Science*, 126(2):183–235, 1994.
- [3] Baier, Christel and Katoen, Joost-Pieter. *Principles of Model Checking*. MIT Press, Cambridge, MA, USA, 2008.
- [4] Bengtsson, Johan and Yi, Wang. Timed automata: Semantics, algorithms and tools. In *Lectures on Concurrency and Petri Nets*, volume 3098 of *Lecture Notes in Computer Science*, pages 87–124. Springer, 2004.
- [5] Čerāns, Kārlis. Decidability of bisimulation equivalences for parallel timer processes. In *Computer Aided Verification*, volume 663 of *Lecture Notes in Computer Science*, pages 302–315. Springer, 1993.
- [6] Clarke, Edmund M., Grumberg, Orna, and Peled, Doron A. *Model Checking*. MIT Press, Cambridge, MA, USA, 1999.
- [7] European Space Agency. Ariane 501: Presentation of inquiry board report. https://www.esa.int/Newsroom/Press_Releases/Ariane_501_-_Presentation_of_Inquiry_Board_report, 1996. Published 23 July 1996; accessed 9 September 2026.
- [8] Henzinger, Thomas A., Nicollin, Xavier, Sifakis, Joseph, and Yovine, Sergio. Symbolic model checking for real-time systems. *Information and Computation*, 111(2):193–244, 1994.
- [9] Hopcroft, John E., Motwani, Rajeev, and Ullman, Jeffrey D. *Introduction to Automata Theory, Languages, and Computation*. Addison-Wesley, 3rd edition, 2006.

- [10] Lieb, Alexander, Göttmann, Hendrik, Lochau, Malte, Luthmann, Lars, and Schürr, Andy. Checking timed bisimilarity with virtual clocks. *Fundamenta Informaticae*, 185(1), 2025. Article 2102; arXiv:2412.15799.
- [11] Meta Engineering. Open-sourcing facebook infer: Identify bugs before you ship. <https://engineering.fb.com/2015/06/11/developer-tools/open-sourcing-facebook-infer-identify-bugs-before-you-ship/>, 2015. Published 11 June 2015; accessed 9 September 2026.
- [12] Mobed, Amirparsa. Afra weak timed reduction. <https://github.com/ParsaM0BB/AfraWeakTimedReduction>, 2026. Version 1.0.0; thesis snapshot a344925 and recorded evaluation revision 3d63904; accessed 9 September 2026.
- [13] Newcombe, Chris, Rath, Tim, Zhang, Fan, Munteanu, Bogdan, Brooker, Marc, and Deardeuff, Michael. How amazon web services uses formal methods. *Communications of the ACM*, 58(4):66–73, 2015.
- [14] Rebeca Research Group. Afra: An integrated environment for modeling and verifying rebeca family designs. <https://rebeca-lang.org/alltools/Afra>, 2026. Accessed 9 September 2026.
- [15] Rebeca Research Group. Afra source repository. <https://github.com/rebeca-lang/org.rebecalang.afra>, 2026. Source contract examined at commit ed3caf6; accessed 9 September 2026.
- [16] Rebeca Research Group. Rebeca compiler source repository. <https://github.com/rebeca-lang/org.rebecalang.compiler>, 2026. Toolchain configuration examined at commit dc663e0; accessed 9 September 2026.
- [17] Rebeca Research Group. Rebeca model checker source repository. <https://github.com/rebeca-lang/org.rebecalang.rmc>, 2026. TTS emitter examined at commit cab60b1; accessed 9 September 2026.
- [18] Rebeca Research Group. Rebeca model checker source repository. <https://github.com/rebeca-lang/org.rebecalang.modelchecker>, 2026. Toolchain configuration examined at commit fb4197f; accessed 9 September 2026.
- [19] Rebeca Research Group. Rebeca modeling language: Official github organization. <https://github.com/rebeca-lang>, 2026. Accessed 9 September 2026.

- [20] Rebeca Research Group. Rebeca state-space analysis source repository. <https://github.com/rebeca-lang/org.rebecalang.statespaceanalysis>, 2026. State-space reader examined at commit b2b0e23; accessed 9 September 2026.
- [21] Rebeca Research Group. Rebeca state-space transformer source repository. <https://github.com/rebeca-lang/org.rebecalang.statespacetransformer>, 2026. State-space reader examined at commit 02fbef1; accessed 9 September 2026.
- [22] Reynier, Pierre-Alain and Sangnier, Arnaud. Weak time petri nets strike back! In *CONCUR 2009 – Concurrency Theory*, volume 5710 of *Lecture Notes in Computer Science*, pages 557–571. Springer, 2009.
- [23] Reynisson, Arni Hermann, Sirjani, Marjan, Aceto, Luca, Cimini, Matteo, Jafari, Ali, Ingólfssdóttir, Anna, and Sigurdarson, Steinar Hugi. Modelling and simulation of asynchronous real-time systems using timed rebeca. *Science of Computer Programming*, 89:41–68, 2014.
- [24] Sirjani, Marjan. *Formal Specification and Verification of Concurrent and Reactive Systems*. PhD thesis, Sharif University of Technology, Tehran, Iran, June 2004.
- [25] Sirjani, Marjan, Movaghar, Ali, Shali, Amin, and de Boer, Frank S. Modeling and verification of reactive systems using rebeca. *Fundamenta Informaticae*, 63(4):385–410, 2004.
- [26] Swamy, Nikhil. Project everest: Advancing the science of program proof. Microsoft Research, <https://www.microsoft.com/en-us/research/blog/project-everest-advancing-the-science-of-program-proof/>, 2019. Published 5 September 2019; accessed 9 September 2026.
- [27] United States General Accounting Office. Patriot missile defense: Software problem led to system failure at dhahran, saudi arabia. Technical Report IMTEC-92-26, United States General Accounting Office, 1992. <https://www.gao.gov/products/imtec-92-26>.

واژه‌نامه فارسی به انگلیسی

الف

ر

Command-Line Interface . . . رابط خط فرمان	End-to-End Testing آزمون انتها به انتها
Rebeca ربکا	Afra افرا
Timed Rebeca ربکای زمان‌دار	Partition افراز
Trace رد	State-Space Explosion . . انفجار فضای حالت
Formal Method روش صوری	

پ

س

Labeled Transition . . . سامانه انتقال برچسب‌دار	Partition Refinement پالایش افراز
System	Formal Refinement پالایش صوری
Timed Labeled Transition System	Hiding پنهان‌سازی

ت

Quotient Transition . . سامانه انتقال خارج‌قسمتی	Formal Development توسعه صوری
System	
Real-Time System سیستم بلادرنگ	
Reactive System سیستم واکنشی	

ج

ش

Bisimulation شبیه‌سازی دوسویه	Time Additivity جمع‌پذیری زمان
---	--

د

ک

Internal Action کنش داخلی	Formal Verification درستی‌یابی صوری
	Software Verification درستی‌یابی نرم‌افزار

سوم

م

مدل کنشگر Actor Model

مشخصه‌نویسی صوری . . Formal Specification

و

وارسی مدل Model Checking

واژه‌نامه انگلیسی به فارسی

H

Hiding پنهان‌سازی

I

Internal Action کنش داخلی

L

Labeled Transition سامانه انتقال برچسب‌دار
System

M

Model Checking واریسی مدل

P

Partition افراز

Partition Refinement پالایش افراز

Q

Quotient Transition سامانه انتقال خارج‌قسمتی

A

Actor Model مدل کشگر

Afra افرا

B

Bisimulation شبیه‌سازی دوسویه

C

Command-Line Interface رابط خط فرمان

E

End-to-End Testing آزمون انتها‌به‌انتها

F

Formal Development توسعه صوری

Formal Verification درستی‌یابی صوری

Formal Method روش صوری

Formal Refinement پالایش صوری

Formal Specification مشخصه‌نویسی صوری

System

R

Reactive System سیستم واکنشی
Real-Time System سیستم بلادرنگ
Rebeca ربکا

S

Software Verification درستی‌یابی نرم‌افزار
State-Space Explosion . . انفجار فضای حالت

T

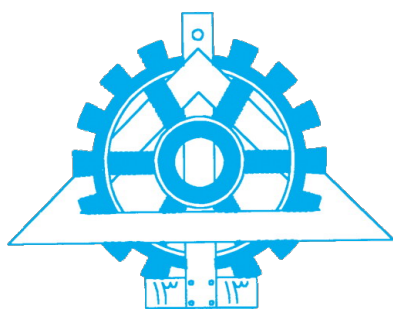
Time Additivity جمع‌پذیری زمان
Timed Labeled . . سامانه انتقال برچسب‌دار زمان‌دار
Transition System
Timed Rebeca ربکای زمان‌دار
Trace رد

Abstract

Model checking automatically verifies desired properties by exploring a system's state space. In concurrent real-time models, combinations of component schedules, message queues, and timing constraints can cause rapid state-space growth and increase the cost of analysis. Model reduction offers a way to control this problem when it preserves the behavior under investigation.

This project investigates and implements the reduction of timed Rebeca semantic models using weak timed bisimulation. The method hides interactions outside the selected observation level and groups states that cannot be distinguished by observable behavior and elapsed time. A smaller model is then constructed from the resulting classes. Evaluation on the available examples indicates that the method can reduce model size while preserving the selected behavioral criterion. The project provides a basis for incorporating this form of reduction into the analysis of Rebeca models in Afra.

Keywords Rebeca, Afra, Weak Timed Bisimulation, Formal Methods, Model Checking



University of Tehran
College of Engineering
School of Electrical and
Computer Engineering
Department of Computer
Engineering



Reduction of Rebeca Semantic Models Using Weak Timed Bisimulation

Bachelor of Science Thesis in Computer Engineering

By:

Amirparsa Mobed

Supervisor:

Dr. Fatemeh Ghassemi Esfahani

September 2026